The Evolution of Large Language Models for Intelligent Information Systems: A Technical Survey and New Perspectives

ANONYMOUS AUTHOR(S), Affiliation withheld for review, Anonymous

Large language models (LLMs) are becoming a core reasoning and interaction layer of AI-based intelligent information systems. Chatbots provide conversational information access; AI search and retrieval-augmented generation (RAG) combine retrieval, ranking, evidence synthesis, and generation; LLM-based recommenders support intent understanding, explanation, and interactive refinement; multimodal assistants operate over heterogeneous information; and agents extend information systems into loops that retrieve, reason, act, and verify. These systems raise a common question: how should models, data, retrieval, memory, tools, and infrastructure be co-designed to improve information access and use?

This survey answers that question through a technique-centric and longitudinal review of LLM evolution. Rather than organizing models by vendor, we trace the technical transitions in architecture, pre-training, post-training and reasoning, long context and memory, multimodality, applications, model training and inference, and evaluation. We use industrial model families as evidence of adoption and scale, and connect model-level advances to information-system functions including search, RAG, recommendation, conversational access, personalization, multimodal retrieval, decision support, and agentic workflow automation.

Three conclusions emerge. First, LLM capability is increasingly an end-to-end system property: retrieval quality, context and memory, post-training, serving design, and tool access can matter as much as base-model scale. Second, the relevant optimization target is a utility frontier over quality, latency, cost, provenance, safety, and interaction success rather than model accuracy alone. Third, interactive capability increasingly depends on successful trajectories and on how effectively a system orchestrates parametric knowledge, retrieval, memory, verification, tools, and external environments rather than on checkpoint scale alone. We conclude with research directions for grounded search and recommendation, persistent memory, trustworthy multimodal access, efficient reasoning, secure tool use, and reliable long-horizon information systems.

CCS Concepts: • **Computing methodologies** → **Natural language processing**; **Machine learning**; • **Information systems** → *Information retrieval*; *Information systems applications*; • **Computer systems organization** → *Cloud computing*.

Additional Key Words and Phrases: large language models, intelligent information systems, AI-based information systems, information retrieval, retrieval-augmented generation, recommendation, chatbots, agents, multimodal models, transformer, pre-training, post-training, reasoning, model training and inference, evaluation, survey

ACM Reference Format:

Anonymous Author(s). 2026. The Evolution of Large Language Models for Intelligent Information Systems: A Technical Survey and New Perspectives. *ACM Trans. Inf. Syst.* 1, 1 (September 2026), 60 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Large language models (LLMs) have moved beyond the role of stand-alone text generators. They increasingly operate as the semantic reasoning and interaction layer of *intelligent information systems*: systems that acquire information, retrieve and rank evidence, integrate heterogeneous sources, support decisions, communicate results, and increasingly act on external environments. Chatbots expose information through dialogue; AI search and retrieval-augmented generation (RAG) combine retrieval with synthesis; LLM-enhanced recommenders interpret intent and explain ranked choices; coding assistants navigate repository-scale information; multimodal assistants integrate text, images, audio, video, and interface state; and agents turn information access into iterative loops of observation, retrieval, reasoning,

Author's Contact Information: Anonymous Author(s), Affiliation withheld for review, Anonymous, Anonymous.

action, and verification. This convergence makes the evolution of LLMs directly relevant to the central concerns of information systems: effective access, ranking, interaction, personalization, provenance, efficiency, and trust.

This perspective is especially timely for information retrieval and related systems research. Recent surveys in ACM Transactions on Information Systems have examined how language models reshape retrieval pipelines, generative retrieval, and GraphRAG [68, 98, 156]. Those works establish that LLMs are already transforming how information is indexed, retrieved, and presented. Our focus is broader and complementary: we ask how the *entire technical stack*—architecture, pre-training, post-training, context and memory, multimodality, applications, training and serving infrastructure, and evaluation—evolved into the substrate on which modern intelligent information systems are built. The objective is therefore not another catalog of model families, nor a survey restricted to one information-access component. It is an explanation of the technical transitions that changed what information systems can do, what they cost, and how they should be evaluated.

1.1 LLMs as a Systems Layer for Information Access and Action

An LLM-centered information system should not be equated with the LLM alone. A deployed search assistant may include a query rewriter, retriever, reranker, generator, cache, user profile, safety layer, and provenance store. A recommender may combine collaborative or content-based retrieval with an LLM for intent understanding, explanation, or conversational refinement. An agent may rely on search indexes, databases, browser tools, code executors, persistent memory, and authorization policies. In each case, the LLM is one component in a larger information-processing pipeline.

This distinction matters because many of the most important improvements now arise from *co-design across components*. Better retrieval can outperform a larger generator when knowledge is fresh or domain-specific. More context can reduce retrieval calls but increase memory and latency. Stronger post-training can improve evidence use without changing pre-training scale. Multi-query attention, grouped-query attention, or latent-attention mechanisms can make long information sessions economical by reducing KV-cache cost. Tool access can compensate for missing parametric knowledge, but introduces security and verification requirements. Serving systems determine whether long-context reasoning, retrieval, and agents remain responsive under real workloads. The practical unit of analysis is therefore an *information workflow* implemented by a model–retrieval–memory–tool–infrastructure stack.

Figure 1 summarizes this view. Information enters through documents, databases, user histories, multimodal content, or external environments. Retrieval, ranking, recommendation, and memory select a working set. The LLM interprets, integrates, reasons, and generates. Interfaces expose answers, rankings, explanations, or actions. Feedback then updates user state, memory, retrieval, or subsequent decisions. The figure also clarifies the boundary of our claim: LLMs increasingly organize the semantic and interactive core of the pipeline, but conventional information-system components remain essential for accuracy, freshness, control, and auditability.

1.2 Why a Technique-Centric Evolutionary Survey?

The LLM literature is often organized either by model family or by a single technical subfield. Both views are useful, but they obscure the causal sequence by which one advance creates the next bottleneck. Transformer scaling increased capability but made distributed training and memory central constraints. Compute-optimal training shifted attention from parameter count to data allocation. RLHF and later preference optimization separated behavioral adaptation from pre-training. Long context increased information bandwidth but exposed quadratic attention and KV-cache costs. MoE separated stored capacity from active compute while increasing communication complexity. Reasoning models moved

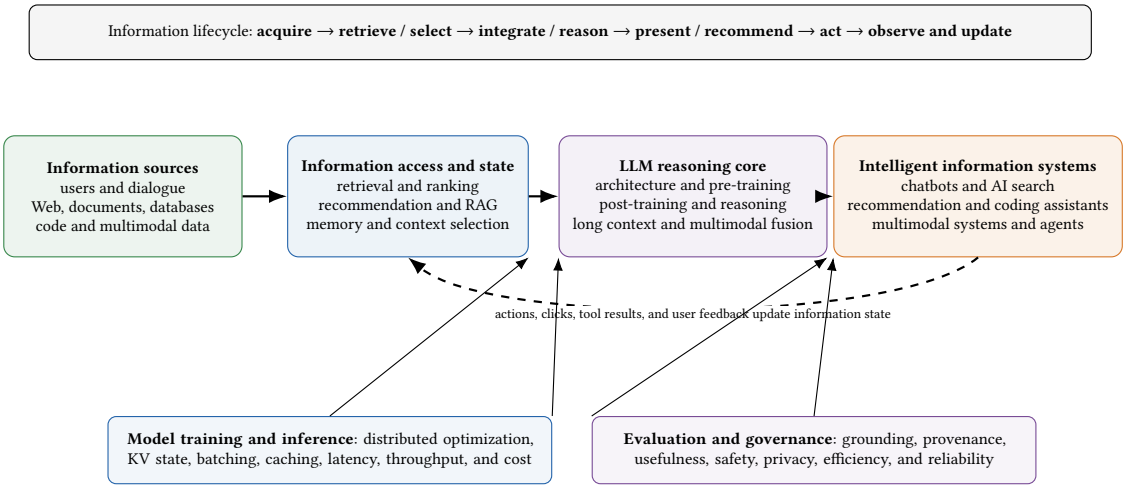


Fig. 1. LLM-centered AI-based intelligent information systems. LLMs increasingly operate inside an end-to-end information lifecycle rather than as standalone text generators. Information sources feed retrieval, ranking, recommendation, RAG, memory, and context-selection components; the LLM integrates and reasons over the resulting state; applications expose the capability through chat, AI search, recommendation, coding, multimodal interfaces, and agents; and subsequent actions or user feedback update the information state. Model training/inference and evaluation/governance support the entire lifecycle.

substantial computation to inference time. Agents then extended the optimization target from a response to a trajectory through an environment.

These transitions are particularly important for information systems because they change the feasible operating point of retrieval, recommendation, interaction, and automation. A technique-centric view also remains more stable than a vendor-centric one: MHA, RoPE, GQA, MoE, DPO, GRPO, FlashAttention, PagedAttention, tensor parallelism, and speculative decoding persist across organizations and model generations. Industrial models are therefore used in this survey as evidence of adoption, scale, and combination of techniques rather than as independent product histories.

Several broad surveys already provide canonical taxonomies of LLMs and multimodal LLMs [144, 150], while more focused surveys address post-training [127], RAG [34], recommendation [140], generative retrieval [68], LLMs for IR [156], and GraphRAG [98]. The gap we target is the cross-layer evolutionary connection among these areas. We study not only *what* techniques exist, but *why they emerged, which bottleneck they moved, how they interact, and what those interactions imply for intelligent information systems.*

1.3 Research Questions

The survey is organized around four research questions. **RQ1: Technical evolution.** Which architectural, data, post-training, multimodal, and systems transitions most strongly changed the capability and cost frontier of LLMs? **RQ2: Information-system consequence.** How do those transitions affect retrieval, ranking, recommendation, evidence integration, personalization, multimodal access, and agentic action? **RQ3: Co-design and trade-offs.** Which resources move across layers—training compute, active inference compute, memory, context, retrieval, and environment interaction—and when can one resource substitute for another? **RQ4: Evaluation.** How should evaluation change when the object under study is an end-to-end information service rather than an isolated foundation model?

Table 1. How LLM technical advances improve AI-based intelligent information systems. The mapping makes explicit that many LLM techniques should be evaluated by the information-system capability they enable, not only by language-model loss or isolated benchmark accuracy.

LLM technical layer	Information-system function	Representative applications	System-level benefit / design question
Architecture (GQA/MLA, MoE, efficient attention)	Efficient representation and processing of large information states	conversational search, long-document QA, high-throughput recommendation, agent memory	How much information can be processed per unit of latency, memory, and cost?
Pre-training and data curation	General semantic representation, domain coverage, multilingual and code knowledge	semantic search, recommendation, enterprise knowledge access, coding assistants	Which information domains and user populations are represented, and with what provenance or bias?
Post-training, reasoning, and preference optimization	Interaction policy, task following, evidence integration, decision support	chatbots, research assistants, decision-support interfaces, reasoning search	How reliably does the system transform retrieved information into useful, controllable decisions?
Long context, retrieval, and memory	Selection, persistence, and reuse of relevant external or historical information	RAG, conversational search, personalized assistants, repository-scale coding	Where should information live: parameters, context, index, or persistent memory?
Multimodal LLMs	Access to heterogeneous information objects and interfaces	document intelligence, visual search, speech assistants, UI understanding	How can text, images, audio, video, and interface state be jointly retrieved, grounded, and explained?
Model training and inference	Scalable and economical operation of intelligent information services	high-QPS search assistants, real-time chat, multi-tenant enterprise systems, agent platforms	What quality–latency–throughput–memory trade-off is sustainable for an information service?
Agents and tools	Closed-loop information seeking, action, verification, and workflow execution	web research, coding agents, business workflow automation, autonomous information gathering	Can the system recover from bad evidence or actions and preserve authorization, provenance, and state?
Evaluation and governance	End-to-end measurement of information quality, usefulness, risk, and efficiency	search/RAG evaluation, recommendation, human preference, tool-use and agent evaluation	Does the complete information system deliver grounded, useful outcomes under explicit resource and safety constraints?

These questions impose a stronger structure than a chronological catalog. Each technical section reconstructs an evolutionary sequence, identifies the bottleneck being addressed, and ends by connecting the technique to the design of intelligent information systems. The later synthesis then asks what can be generalized across topics.

1.4 Contributions of This Survey

This survey makes five contributions. First, it provides a *longitudinal, technique-centric account* of LLM evolution across architecture, pre-training, post-training, long context and memory, multimodality, applications, training and inference, and evaluation. Second, it develops an *information-systems interpretation* of these transitions, connecting model-level advances to search, RAG, recommendation, conversational access, personalization, multimodal information processing, decision support, and agents. Third, it analyzes *cross-layer trade-offs*: changes that improve one layer often move cost or complexity elsewhere, for example from model weights to communication, from retrieval calls to context bandwidth, or from pre-training to test-time reasoning. Fourth, it treats industrial reports as evidence without conflating product claims with reproducible scientific results, using an explicit evidence hierarchy and an auditable literature-selection protocol. Fifth, it proposes synthesis perspectives centered on successful trajectories, cross-component orchestration,

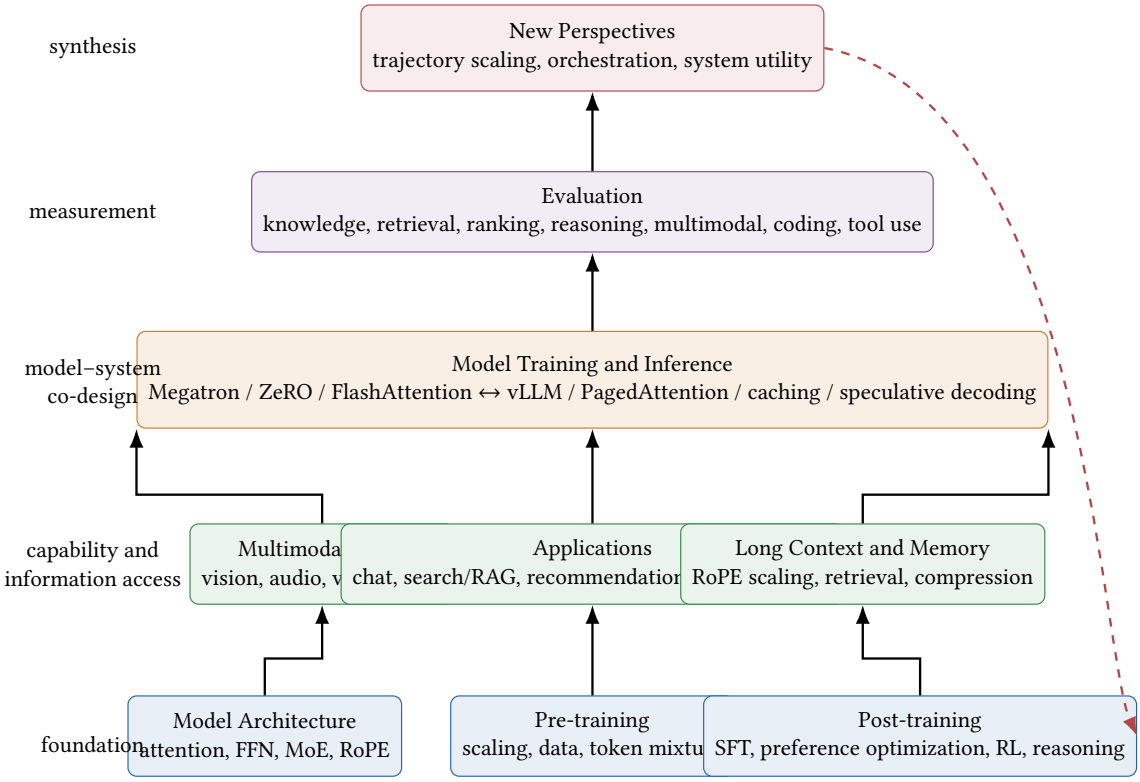


Fig. 2. Bird's-eye view of the technical landscape surveyed in this paper. The revised structure treats training and inference as one co-designed systems layer and makes information access—search, RAG, ranking, recommendation, and retrieval agents—a central application and evaluation thread.

and end-to-end information-system utility. Together they motivate an optimization target over information quality, task success, latency, cost, provenance, and risk.

The survey is intentionally not a leaderboard. Benchmark values depend on prompting, retrieval access, tool use, sampling, inference budget, contamination, and undocumented post-training. Our emphasis is instead on *technical transition and system consequence*: which ideas changed the design space, which bottlenecks they addressed, which new bottlenecks they created, and how they propagated into information-system practice.

1.5 Roadmap

Figure 2 provides a roadmap. Section 2 defines the corpus, evidence standards, and comparison dimensions. Section 3 reconstructs the cross-topic timeline. Sections 4–10 then examine architecture, pre-training, post-training, multimodality, applications, model training and inference, and evaluation, with information-system implications integrated throughout. Section 11 develops the trajectory-scaling, orchestration, and end-to-end utility perspectives, and Sections 12 and 13 discuss open problems and conclusions.

2 Scope, Evidence, and Survey Methodology

Surveying industrial LLM evolution is methodologically difficult because evidence quality is highly uneven. Open-weight projects may disclose architecture, token counts, training recipes, and ablations; closed commercial systems may disclose only model cards, safety reports, or high-level capability statements. Even apparently simple quantities are not always comparable: some reports state total parameters while others emphasize active parameters, context-window claims may refer to trained length or supported inference length, and benchmark results may use different prompts, tools, retrieval access, or reasoning budgets. We therefore separate *technical evidence* from *product claims*, avoid inferring undisclosed details, and make the review protocol explicit.

2.1 Review Scope and Questions

The review covers the period from the emergence of the Transformer in 2017 through September 2026, while retaining a small number of earlier references required for optimization, evaluation, and information-retrieval background. The scope is intentionally cross-layer: architecture, pre-training, post-training and reasoning, long context and memory, multimodality, applications, distributed training, inference systems, and evaluation. Information retrieval, RAG, recommendation, conversational access, and agents are treated not as peripheral applications but as information-system settings in which these technical choices become observable.

The literature is coded against the four research questions introduced in Section 1: technical evolution (RQ1), information-system consequences (RQ2), cross-layer trade-offs (RQ3), and end-to-end evaluation (RQ4). This coding discourages a paper-by-paper narrative. A source is retained because it helps explain a technical transition, adoption pattern, system trade-off, or evaluation shift relevant to at least one research question.

2.2 Corpus Construction and Screening

The corpus combines four source classes. *Foundational research* covers Transformer architecture, optimization, scaling, alignment, multimodality, retrieval, recommendation, agents, and evaluation. *Primary model reports* document industrial LLM families when sufficient technical detail is available. *Systems research* covers distributed training, attention kernels, serving, caching, scheduling, and disaggregation. *Benchmark and survey literature* provides evaluation protocols and high-level organization of rapidly growing subfields. The curated *Awesome-LLM-papers* repository is used as a discovery index because it spans algorithms, systems, SFT, RL, multimodality, MoE, quantization, agents, coding, and evaluation [41]; substantive claims are cited to primary sources whenever possible.

We use an auditable narrative-review protocol rather than claiming a PRISMA-style exhaustive database search. The seed corpus is expanded through backward and forward snowballing, primary technical reports, and topic-specific surveys. A source is included when it satisfies at least one of three criteria: (1) it introduced or strongly popularized a reusable technique that appears in later systems; (2) it provides enough implementation, analysis, or ablation detail to explain a technical effect; or (3) it documents industrial-scale adoption that materially changes the capability or cost frontier. We exclude routine product updates without technical disclosure, duplicate reports, unverifiable parameter estimates, and marketing-only claims. Figure 3 records the workflow and Table 2 reports the temporal composition of the cited corpus.

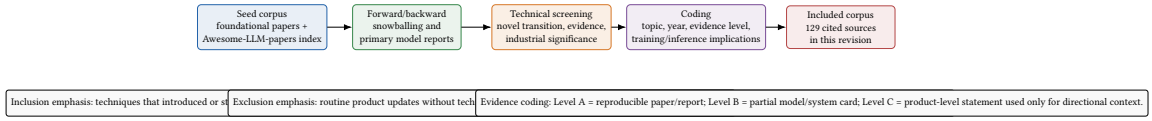


Fig. 3. Literature construction and screening protocol used in this survey. The process is intentionally auditable rather than presented as an exhaustive database search: the corpus is seeded by foundational work and a curated LLM index, expanded by snowballing and primary reports, screened for technical significance, and coded by topic, time, and evidence level.

Table 2. Audit of the literature cited in this manuscript. Counts describe the retained cited corpus, not the full LLM literature.

Publication period	Cited sources	Role in the survey
Before 2017	4	Background required for optimization, sequence modeling, and evaluation foundations.
2017–2020	21	Transformer foundations, early GPT scaling, retrieval, distributed optimization, and scaling laws.
2021–2022	19	Compute-optimal scaling, RLHF, MoE maturation, FlashAttention, chain-of-thought, and generative retrieval.
2023–2024	61	Rapid diversification: open-weight LLMs, DPO, long context, multimodality, RAG, serving systems, agents, and recommendation.
2025–2026	24	Sparse industrial architectures, reasoning RL, omni-modal models, agentic post-training, recent information-access surveys, and system co-design.
Total	129	The retained corpus is deliberately recent-heavy while preserving the earlier work needed to reconstruct technical evolution.

2.3 Positioning Relative to Prior Surveys

The survey is deliberately positioned against both broad LLM reviews and recent information-access surveys. Zhao et al. provide a canonical broad taxonomy of pre-training, adaptation, utilization, and evaluation [150]; Yin et al. organize multimodal LLM architectures, data, tasks, and evaluation [144]; and Tie et al. focus on post-training and reasoning models [127]. Other surveys focus on RAG [34] and LLM-based recommendation [140]. In ACM Transactions on Information Systems, Li et al. survey generative information retrieval [68], Zhu et al. survey LLMs across the IR pipeline [156], and Peng et al. systematize GraphRAG [98].

Our contribution is complementary rather than duplicative. The unit of analysis here is the *evolving LLM stack and the bottlenecks it moves across layers*. We connect architecture and data choices to post-training, context, retrieval, serving, and evaluation; we use information systems as the deployment lens; and we ask how technical changes affect end-to-end utility rather than only one stage of the retrieval or generation pipeline. Table 3 makes these differences explicit.

2.4 Temporal and Technical Organization

The survey uses two coordinates simultaneously. The first is *technical topic*: architecture, pre-training, post-training, multimodality, applications, training and inference, and evaluation. The second is *evolutionary stage*: foundation, scaling, industrialization, and current frontier. The same model can therefore appear in multiple sections when it provides evidence for different transitions. DeepSeek-V2, for example, is relevant to both MoE and MLA; Gemini 1.5 to long context and multimodality; Qwen3 to controllable reasoning; and Kimi/Mooncake to long-context modeling and serving.

This organization is chosen to expose dependency rather than chronology alone. A technique is discussed where it changes the design space, and later sections trace the consequences of that change. For instance, GQA first appears as an

Table 3. Positioning relative to representative LLM and information-access surveys. Prior work typically studies either the broad LLM stack or one information-access subfield; this paper focuses on the longitudinal connections among model techniques, systems, and intelligent information-system functions.

Survey	Primary scope	Main organizing lens	Complementary focus of this paper
Zhao et al. [150]	General LLMs	Pre-training, adaptation, utilization, evaluation, resources	Longitudinal evolution through 2026; stronger systems and information-system interpretation.
Yin et al. [144]	Multimodal LLMs	Architecture, data, tasks, evaluation	Places multimodality within the same evolution as post-training, context, agents, and serving infrastructure.
Tie et al. [127]	Post-training and reasoning	Alignment and reasoning methods	Connects post-training to retrieval, test-time cost, serving, applications, and end-to-end evaluation.
Gao et al. [34]	Retrieval-augmented generation	RAG pipeline and methods	Treats RAG as one information-system pattern shaped by the broader model, context, memory, and inference stack.
Wu et al. [140]	LLM-based recommendation	Roles of LLMs in recommender systems	Connects recommendation to the same architecture, data, post-training, personalization, and cost trade-offs as search and agents.
Li et al. [68]	Generative information retrieval	Generative retrieval and generation-oriented IR	Explains how the underlying LLM stack and serving economics enable or constrain generative information access.
Zhu et al. [156]	LLMs for information retrieval	Query rewriting, retrieval, reranking, reading, search agents	Extends beyond IR components to the full LLM lifecycle and its interaction with recommendation, multimodality, agents, and systems.
Peng et al. [98]	GraphRAG	Graph indexing, retrieval, and generation	Uses GraphRAG as one instance of a broader shift toward external memory, structured information, and model–retrieval co-design.
This paper	LLM evolution for intelligent information systems	Technical transition, bottleneck movement, and cross-layer co-design	Integrates architecture, data, post-training, information access, multimodality, training/inference, evaluation, and new scaling perspectives in one longitudinal framework.

architectural change, then reappears as an inference-memory advantage; long-context training appears in post-training, but its value is interpreted through retrieval, memory, and serving cost.

2.5 Dimensions of Comparison

Across topics we compare seven dimensions: *information-system function*, *capability*, *compute*, *memory*, *data*, *control*, and *reproducibility*. Information-system function asks which stage of the information lifecycle is improved—retrieval, ranking, recommendation, evidence integration, interaction, memory, decision support, or action. Capability asks what behavior becomes possible. Compute asks where FLOPs move—pre-training, active parameters, or test time. Memory asks which state grows—parameters, optimizer state, activations, context, or KV cache. Data asks what supervision or coverage is required. Control asks whether users or systems can allocate effort, tools, modalities, or retrieval. Reproducibility asks how much of the claimed effect can be independently verified.

These dimensions prevent model quality from being treated as independent of the service in which the model operates. GQA and MLA both reduce inference-state cost but do so through different parameterizations. Position interpolation, sparse attention, retrieval, and persistent memory all address information availability, but trade off context bandwidth, freshness, latency, and implementation complexity. DPO, PPO-based RLHF, GRPO, and verifier-based RL all adapt behavior, but require different preference, sampling, and verification pipelines.

2.6 Evidence Levels and Uncertainty

We distinguish three evidence levels. **Level A** evidence comes from peer-reviewed papers or open technical reports with sufficient methodological detail for independent implementation or close reproduction. **Level B** evidence comes from model cards, system cards, or vendor reports with partial technical disclosure. **Level C** evidence consists of product-level statements for which essential training or evaluation details are unavailable. Level C evidence can indicate industry direction but is not used to establish a causal technical conclusion.

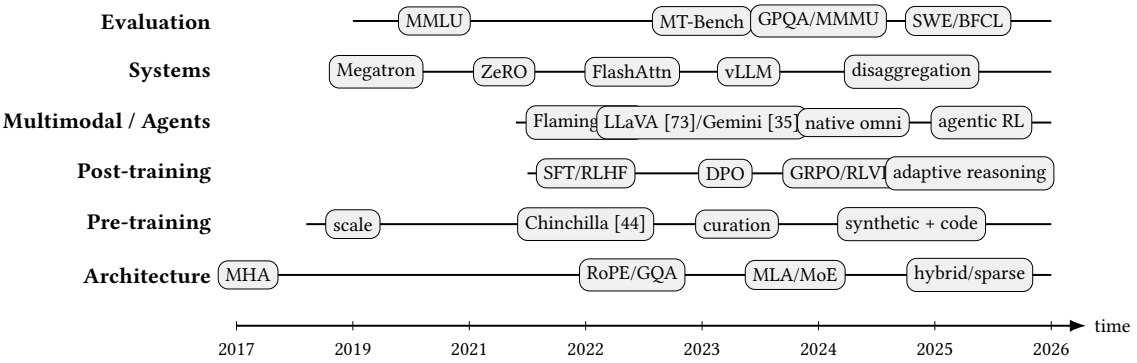


Fig. 4. A technique-centric timeline of major LLM transitions. The tracks overlap: later systems combine techniques introduced in different periods.

This hierarchy is particularly important for closed frontier models. A strong benchmark result establishes observable capability, but it does not reveal whether the gain came from architecture, data, post-training, tool use, retrieval, or test-time compute. Where details are undisclosed, we describe only the observable system behavior and avoid reverse-engineering hidden parameterization from public outputs.

2.7 Limits of the Review

Three limitations should be explicit. First, the literature is expanding too rapidly for any static survey to be exhaustive; our goal is explanatory coverage of influential transitions rather than complete enumeration. Second, industrial disclosure is asymmetric, so open systems are easier to analyze causally than closed ones. Third, publication and benchmark visibility can bias the corpus toward English-language and widely benchmarked systems. We mitigate these limitations through evidence labels, primary-source preference, and technique-centric comparison, but they cannot be eliminated entirely.

A technique-centric organization is nevertheless comparatively stable. Product names and release cadences change quickly, whereas MHA, RoPE, GQA, SwiGLU, MoE, RLHF, DPO, GRPO, FlashAttention, PagedAttention, and tensor/pipeline/expert parallelism persist across generations. This stability lets the survey focus on reusable mechanisms and their consequences for information-system design.

3 A Cross-Topic View of LLM Evolution

The technical history of LLMs is not a sequence of cleanly separated eras; it is a set of overlapping bottleneck shifts. Nevertheless, a coarse timeline is useful because many later techniques are responses to constraints created by earlier successes. Figure 4 and Table 4 summarize the central transitions.

3.1 Parameter Scale: From Dense Growth to Sparse Capacity

Parameter count was the most visible scaling axis in the first LLM era, but its meaning changed as architectures became sparse. Figure 5 and Table 5 plot representative models for which parameter counts are publicly disclosed. GPT-1 grew from roughly 0.117B parameters to GPT-2 at 1.5B and GPT-3 at 175B [12, 106, 107]; PaLM reached 540B dense parameters in 2022 [18]. Yet later results such as LLaMA showed that a smaller model trained on a better compute/data

Table 4. Representative technical transitions in LLM evolution. Dates indicate influential publication/adoption periods rather than exclusive invention dates.

Period	Dominant bottleneck	Representative techniques	Shift in the design objective
2017–2020	Representation capacity and scalable optimization	Transformer, causal LM, MHA, dense FFN, large-batch distributed training	From task-specific models to general pre-trained models; quality mainly tracks scale.
2021–2022	Data/compute balance and usable assistant behavior	Chinchilla [44] scaling, instruction tuning, RLHF, ZeRO, Megatron 3D parallelism, FlashAttention	From parameter count alone to compute-optimal training and a distinct post-training stage.
2023–2024	Serving state, openness, long context, multimodality	GQA, RoPE scaling, DPO, LLaVA [73], Gemini [35], PagedAttention, speculative decoding	From a text-only model to a deployable system with efficient decoding and multiple modalities.
2024–2026	Active compute, reasoning cost, trajectories	MoE, MLA, GRPO/RLVR, native multimodality, hybrid attention, agents, disaggregated serving	From monolithic scaling to allocation of sparse capacity, context bandwidth, test-time compute, and environment interaction.

allocation could be competitive, reinforcing the lesson of compute-optimal scaling that parameter count alone is not a sufficient predictor of quality [44, 128].

The newer trend is a separation between *total capacity* and *active compute*. DeepSeek-V2 exposes 236B total parameters but activates about 21B per token; DeepSeek-V3 increases total capacity to 671B with 37B active [25, 26]. Llama 4 Maverick reports 400B total and 17B active parameters, Qwen3-235B-A22B reports 235B total and 22B active, and Kimi K2 reports 1T total with 32B active [60, 78, 141]. The important historical trend is therefore not an indefinitely rising dense-parameter curve. It is a transition from *dense scale* to *sparse stored capacity*, where model quality, memory footprint, communication, and per-token compute must be analyzed separately. Figure 5 therefore restricts itself to models with publicly disclosed parameter scales and places them on a single time axis, while using marker type to distinguish dense models from sparse MoE totals and active-per-token values. This avoids mixing disclosed and undisclosed families while still making the progression from dense scaling to sparse-capacity scaling visually easy to read. Table 5 provides a broader chronology of influential models whose parameter counts are publicly documented.

Table 5. Major industry-recognized LLMs with publicly disclosed parameter scales. The table broadens the chronology beyond Figure 5; models with undisclosed or non-comparable parameter counts are intentionally excluded. "Active" denotes parameters used for one token in an MoE model.

Year	Representative model	Architecture	Parameters	Evolutionary significance
2018	GPT-1 [106]	Dense	0.117B	Establishes generative pre-training as a transferable language-modeling interface.
2019	GPT-2 [107]	Dense	1.5B	Demonstrates strong zero-shot transfer and a sharp scale increase within the GPT family.
2020	GPT-3 [12]	Dense	175B	Makes large-scale few-shot learning and parameter scaling central hypotheses for general-purpose LMs.
2022	MT-NLG [120]	Dense	530B	Couples Megatron tensor/pipeline parallelism with DeepSpeed to demonstrate extreme-scale dense training.
2022	Chinchilla [44]	Dense	70B	Shows that a smaller model trained on substantially more data can outperform much larger under-trained models.
2022	OPT-175B [148]	Dense	175B	Provides a large openly available decoder family and detailed training logs for reproducibility research.
2022	BLOOM [111]	Dense	176B	Establishes a large multilingual open-access model trained through an international collaborative effort.
2022	PaLM [18]	Dense	540B	Represents the high end of dense scaling before sparse capacity becomes a dominant industrial direction.

Continued on next page

Table 5 – continued from previous page

Year	Representative model	Architecture	Parameters	Evolutionary significance
2022–23	GLM-130B [146]	Dense	130B	Demonstrates large bilingual open pre-training and a distinct autoregressive blank-infilling lineage.
2023	LLaMA-65B [128]	Dense	65B	Shows that smaller, better-trained open-weight models can be highly competitive.
2023	Llama 2-70B [129]	Dense	70B	Extends the open Llama ecosystem with chat-oriented post-training and a widely adopted 70B scale.
2023	Falcon-180B [4]	Dense	180B	Represents a major openly released dense model trained on a very large web corpus and large GPU cluster.
2023	Qwen-72B [8]	Dense	72B	Establishes a strong multilingual and tool-oriented industrial model family with broad downstream adoption.
2023	Yi-34B [1]	Dense	34B	Demonstrates strong bilingual open-model performance driven by extensive data engineering and long-context extensions.
2023	Mistral-7B [52]	Dense	7B	Shows that architecture and data efficiency, including GQA and sliding-window attention, can challenge substantially larger models.
2023–24	Mixtral 8×7B [53]	MoE	46.7B total / 12.9B active	Popularizes sparse MoE in the open ecosystem and makes total versus active parameter count a practical distinction.
2024	Gemma 2-27B [38]	Dense	27B	Illustrates strong practical-size open models using local/global attention and distillation-oriented training choices.
2024	Llama 3.1-405B [31]	Dense	405B	Demonstrates continued viability of very large dense open-weight models alongside the emerging MoE frontier.
2024	Qwen2.5-72B [102]	Dense	72B	Extends the Qwen family across general, coding, mathematics, and multilingual tasks with broad open deployment.
2024	DeepSeek-V2 [25]	MoE	236B total / 21B active	Separates stored capacity from per-token compute and pairs sparse FFNs with Multi-head Latent Attention.
2024	DeepSeek-V3 [26]	MoE	671B total / 37B active	Scales sparse capacity while combining low-precision training and auxiliary-loss-free balancing.
2025	Llama 4 Maverick [78]	MoE	400B total / 17B active	Brings sparse activation and native multimodality into the Llama family.
2025	Qwen3-235B-A22B [141]	MoE	235B total / 22B active	Combines sparse capacity with controllable thinking and non-thinking post-training modes.
2025	GLM-4.5 [39]	MoE	355B total / 32B active	Couples sparse scaling with hybrid reasoning and explicit agentic/coding-oriented post-training.
2025	Kimi K2 [60]	MoE	1T total / 32B active	Reaches trillion-parameter stored capacity while keeping per-token activation in the tens of billions.
2026	MiniMax-M2 [82]	MoE	229.9B total / 9.8B active	Pushes the trend toward small active footprints for agentic and coding-oriented models despite large stored capacity.

Table 6. Representative industrial models as evidence points for technical transitions. The purpose is not to rank families but to show cross-cutting adoption. "✓" indicates a prominently documented use or emphasis in the cited generation.

Representative model/generation	MoE / sparse FFN	KV-efficient attention	Long context	Native/strong multimodal	Reasoning/RL emphasis	Agent/tool emphasis	Open-weight evidence
GPT-4/o/5 systems [89, 90, 92]	–	undisclosed	✓	✓	✓	✓	–
Gemini 1.5/2.5 [36, 37]	documented in family	undisclosed	✓	✓	✓	✓	–
Llama 3/4 [31, 78]	✓(Llama 4)	GQA	✓	✓(Llama 4)	partial	tool ecosystem	✓
Qwen2.5/3/Omni [102, 103, 141]	MoE variants	GQA-family designs	✓	✓	✓	✓	✓
DeepSeek-V2/V3/R1 [25, 26, 28]	✓	MLA	✓	partial	✓	coding/tool use	✓
Kimi k1.5/K2 [59, 60]	✓(K2)	efficient/hybrid research	✓	✓	✓	✓	✓
GLM-4.5/5 [39, 40]	✓	efficient attention variants	✓	model-dependent	✓	✓	✓
MiniMax-01/M1/M2 [80–82]	✓	Lightning/hybrid attention	✓	model-dependent	✓	✓	✓

3.2 2017–2020: The Transformer and the Scaling Hypothesis

The Transformer replaced recurrence with self-attention and position-wise feed-forward computation, enabling efficient parallel training and flexible dependency modeling [130]. GPT-1 and GPT-2 established generative pre-training as a transferable interface [106, 107]; GPT-3 then showed that scale could produce strong few-shot behavior without gradient-based task adaptation [12]. The dominant research question became how performance scales with model size, data, and compute. Kaplan et al. formalized empirical scaling laws [56], while systems research such as Megatron-LM showed that scaling required model-parallel training rather than algorithmic changes alone [119].

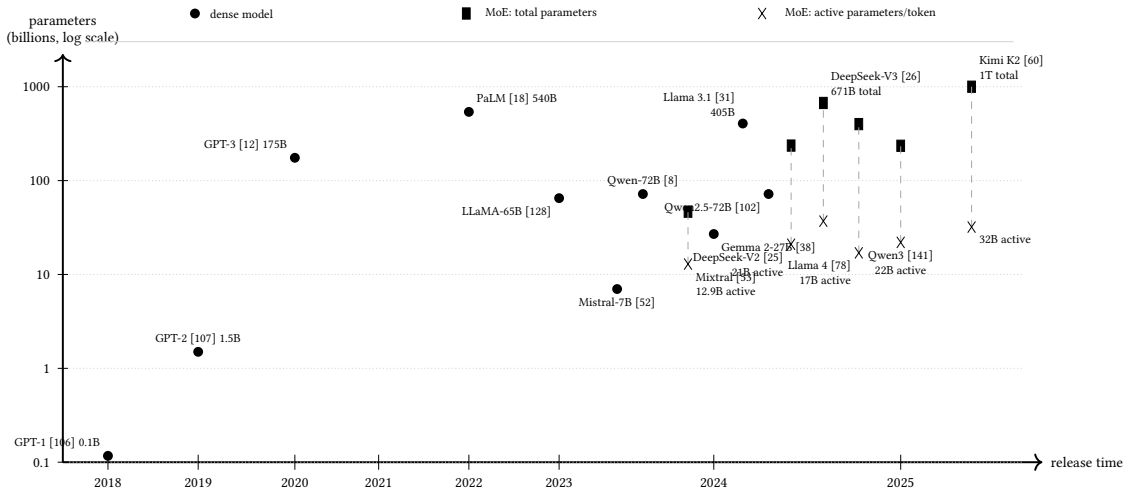


Fig. 5. Important industrial LLMs with publicly reported parameter scales, organized along a single release timeline. Circles denote dense models, filled squares denote total parameters for MoE models, and cross markers denote active parameters per token for MoE models. The single timeline makes the transition from dense scaling to sparse-capacity scaling visible while keeping total and active MoE parameter counts distinct.

Architecturally, this period stabilized the decoder-only Transformer as a general-purpose LLM backbone. However, nearly every major later change can be understood as an attempt to reduce one of its costs: quadratic attention, large feed-forward compute, optimizer-state memory, or autoregressive decoding latency.

3.3 2021–2022: Compute-Optimal Training and the Post-Training Split

The next bottleneck was no longer simply parameter count. Chinchilla argued that model size and token count should be scaled together under a compute budget, demonstrating that smaller models trained on more data could outperform much larger under-trained models [44]. In parallel, instruction tuning and RLHF showed that user-facing quality was not determined by pre-training loss alone. InstructGPT's SFT–reward-model–PPO pipeline established post-training as a separate phase that could dramatically change model behavior without re-learning world knowledge from scratch [93].

System work also accelerated: ZeRO sharded optimizer state and parameters [109], Megatron combined tensor, pipeline, and data parallelism at cluster scale [86], and FlashAttention reframed attention as an IO problem rather than only an arithmetic problem [22].

3.4 2023–2024: Efficiency, Open Ecosystems, Long Context, and Multimodality

By 2023, the field diversified along multiple axes. Llama popularized strong open-weight decoder models with a modernized architecture using RMSNorm, SwiGLU, and RoPE [128]. GQA and related attention variants reduced KV-cache memory for decoding [2, 116]. DPO simplified preference optimization [108]. Flamingo and LLaVA demonstrated alternative routes to visual-language integration [3, 73], while Gemini framed multimodality as a native foundation-model property [35]. Long-context work combined position scaling, data, and attention/system optimizations; Gemini 1.5 made million-token contexts a visible industrial target [36].

The inference stack also matured. vLLM's PagedAttention improved KV-cache utilization and continuous batching [64]; speculative decoding attacked sequential generation latency [14, 65]; and SGLang later generalized prefix/KV reuse for structured LLM programs [152]. These developments made serving strategy part of model design.

3.5 2024–2026: Sparse Capacity, Reasoning RL, Native Multimodality, and Agents

Recent industrial models increasingly combine sparse activation with state-efficient attention. DeepSeek-V2 paired MoE with MLA [25]; DeepSeek-V3 scaled the design with low-precision training and auxiliary-loss-free balancing [26]; Llama 4 adopted MoE and native multimodality [78]; Kimi K2 used a large sparse architecture together with agentic post-training [60]; and MiniMax explored hybrid Lightning Attention for long sequences [80]. The architectural direction is therefore not simply "larger" but "more capacity per unit of active computation and memory traffic."

At the same time, reasoning changed the meaning of inference. Chain-of-thought had already shown that additional generated computation could improve difficult tasks [137]; DeepSeekMath introduced GRPO for mathematical reasoning [115]; DeepSeek-R1 showed that large-scale reinforcement learning with verifiable rewards can produce sophisticated reasoning behavior [28]; Kimi k1.5 similarly emphasized scaling RL and long chain-of-thought [59]. Commercial systems now often expose reasoning effort or "thinking" controls, making inference-time compute an explicit resource dimension.

Finally, agents move the optimization target beyond a single response. ReAct interleaves reasoning with actions [143]; Toolformer trains API-use behavior [112]; software-engineering and function-calling benchmarks evaluate stateful interaction [54, 95]; and recent industrial post-training increasingly includes long tool-use trajectories. The unit of success becomes a completed task trajectory rather than a next-token prediction.

3.6 A Recurrent Pattern: Capability Creates the Next Systems Bottleneck

The timeline reveals a recurrent pattern. Scaling model size creates optimizer-state and communication bottlenecks, motivating ZeRO and Megatron. Long context creates quadratic attention and KV-cache bottlenecks, motivating FlashAttention, MQA/GQA/MLA, sparse attention, and disaggregated serving. Reasoning improves hard-task accuracy but creates long-decoding and verification cost, motivating adaptive budgets and speculative decoding. Agents increase usefulness but create asynchronous environment waits, long-lived state, and evaluation difficulty. Thus the evolution of LLMs is best viewed as a feedback loop between capability innovation and systems bottleneck removal.

3.7 From LLM Evolution to Information-System Evolution

Viewed through an information-systems lens, these phases correspond to a widening of what the system can do with information. Early language models primarily transformed information already present in a prompt. Instruction tuning turned them into usable interactive interfaces. Retrieval and long context connected generation to external and dynamically updated information. Multimodality expanded the addressable information space from text to documents, images, audio, video, and interface state. Reasoning and agents then extended the system from presenting information to selecting evidence, making decisions, invoking tools, and revising actions. The relevant trajectory is therefore not only *model scaling*; it is the evolution from a generative component toward an orchestrator inside an intelligent information system. This interpretation motivates the application and evaluation criteria used throughout the remainder of the survey.

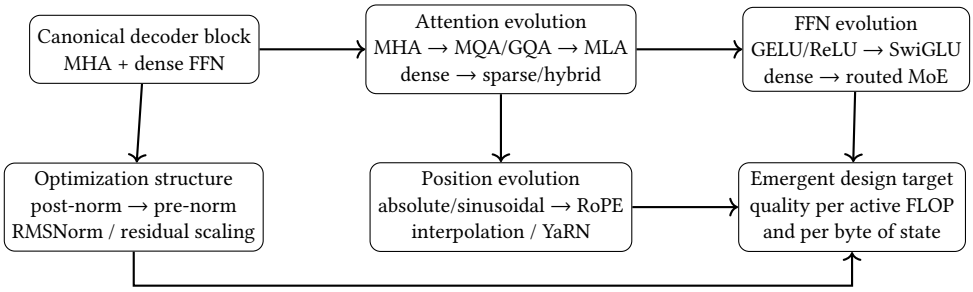


Fig. 6. Architecture evolution can be read as progressive optimization of active compute, state memory, length generalization, and training stability.

Table 7. Evolution of major architectural components in decoder-style LLMs.

Component	Early dominant design	Later evolution	Why the change matters
Attention projections	Multi-head attention (MHA)	MQA, GQA, MLA	Reduces KV-cache memory/bandwidth while retaining many query heads; MLA explicitly compresses attention state.
Attention pattern	Dense softmax attention	IO-aware dense kernels, block sparse, linear/hybrid attention	Makes long context feasible by attacking memory traffic or asymptotic sequence cost.
FFN activation	ReLU/GELU MLP	GLU/SwiGLU	Improves quality/optimization at similar parameter and compute budgets.
FFN capacity	Dense FFN	Top-k routed MoE, fine-grained/shared experts	Decouples total capacity from active per-token compute, at the cost of routing and communication.
Position	Sinusoidal / learned absolute	RoPE, position interpolation, YaRN	Improves relative-position modeling and enables practical context extension.
Normalization	LayerNorm / post-norm	Pre-norm, RMSNorm, residual scaling	Improves stability and reduces overhead for very deep/large models.
Design metric	Parameter count	Active parameters + KV state + bandwidth + stability	Better predicts training and serving economics of modern LLMs.

4 Model Architecture: From the Canonical Transformer to Sparse and State-Efficient LLMs

Although contemporary LLMs differ in scale and training recipe, most still inherit the decoder-only Transformer skeleton: token embeddings, repeated attention and feed-forward blocks, residual connections and normalization, followed by an output projection. Architectural evolution has therefore been evolutionary rather than revolutionary. The major changes target four cost centers: *attention computation*, *attention state*, *feed-forward compute*, and *position/length generalization*. Figure 6 and Table 7 summarize the trajectory. Figure 7 adds a more polished survey-style taxonomy of the main architectural branches and the bottlenecks they target.

4.1 Self-Attention: From MHA to MQA, GQA, and Latent Attention

Multi-head self-attention (MHA) lets each head form its own query, key, and value projections [130]. In pre-training, this design is highly parallel, but autoregressive serving exposes a memory problem: every generated token requires

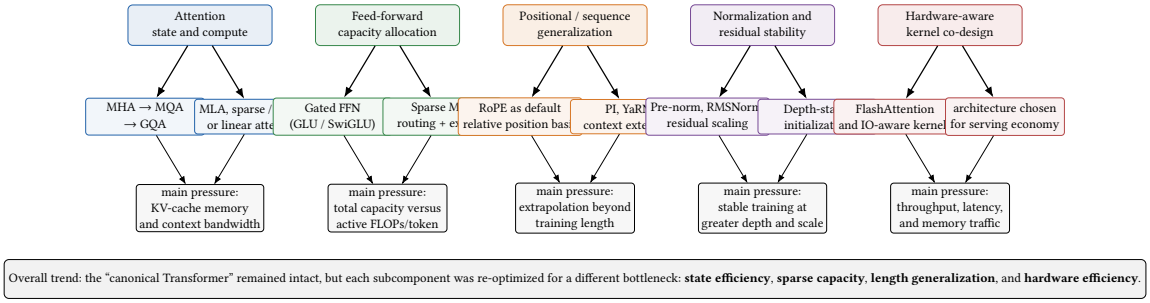


Fig. 7. A survey-style taxonomy of architectural evolution in LLMs. Rather than replacing the Transformer wholesale, industrial models progressively modified a few recurring components. The figure groups these changes by bottleneck and shows how architectural design became inseparable from serving and systems concerns.

storing keys and values for every layer and head. As context length, batch size, and model width grow, the KV cache can dominate accelerator memory.

Multi-query attention (MQA) reduces this state by sharing one set of keys and values across many query heads [116]. The trade-off is reduced representational flexibility. Grouped-query attention (GQA) interpolates between MHA and MQA by sharing K/V within groups of query heads [2]. GQA became a practical default in many open models because it materially reduces KV memory and bandwidth with limited quality loss. Llama 3, for example, uses GQA across model sizes [31]; related designs appear across Qwen and other industrial families.

DeepSeek-V2 introduced a more aggressive state-compression idea, Multi-head Latent Attention (MLA), which compresses key/value representations into a lower-dimensional latent state and reconstructs attention-specific components as needed [25]. The conceptual shift is important: attention design is no longer judged only by training perplexity but by the size and movement of inference state. Later DeepSeek models retain MLA while scaling sparse capacity [26]. This illustrates a broader trend from *compute-centric* attention design toward *state-centric* attention design.

A second attention trend concerns long sequences. Exact dense attention remains attractive because of its quality and simplicity, especially when IO-aware kernels such as FlashAttention reduce hardware overhead [21, 22]. But million-token workloads motivate sparse or hybrid designs. MoBA routes queries to blocks of tokens [77]; Kimi Linear combines linear-attention ideas with expressive gating [61]; MiniMax-01 interleaves Lightning Attention with conventional softmax attention [80, 100]. These designs seek to preserve local or salient interactions while changing the asymptotic or effective cost curve.

4.2 Feed-Forward Networks: From Dense MLPs to Gated Units and MoE

In a standard Transformer, the feed-forward network (FFN) applies two dense projections with a nonlinearity independently to each token. At LLM scale, FFNs account for a large fraction of parameters and FLOPs. The first important evolution was not sparsity but *gating*. GLU-family activations multiply a transformed branch by a gate; SwiGLU became especially common because it improves optimization and quality at comparable compute [117]. Modern Llama-style blocks combine SwiGLU with RMSNorm and pre-normalization [31, 128].

Mixture-of-experts changes the FFN from one dense transformation into a pool of experts with token-dependent routing. Earlier sparsely gated MoE work showed that parameter capacity could grow much faster than per-token computation [118]. Switch Transformers simplified routing to a small number of experts per token and demonstrated

large sparse models at scale [33]. The core economic motivation is straightforward: if a dense FFN has P parameters that all activate on every token, inference compute grows with P; with MoE, total capacity can grow while active parameters stay much smaller.

Industrial MoE designs evolved in several directions. DeepSeekMoE introduced fine-grained expert segmentation and shared experts to improve specialization and reduce redundancy [27]. DeepSeek-V2/V3 scaled this pattern while introducing balancing strategies designed to avoid routing collapse without large auxiliary-loss penalties [23, 25, 26]. Llama 4, Kimi K2, GLM-4.5/5, and MiniMax-M2 also reflect the broader shift toward sparse frontier architectures [39, 40, 60, 78, 82]. The evolution is thus from *sparse capacity as an experimental scaling trick* to *sparse capacity as an industrial default for high-capacity models*.

MoE creates new costs. Routing induces all-to-all communication, expert imbalance can strand compute, and serving low-batch workloads may underutilize experts. Expert parallelism becomes an architectural requirement rather than an optional systems optimization. Consequently, MoE gains depend on network topology, token batching, routing stability, and kernel efficiency as much as on the routing equation itself.

4.3 Positional Encoding: From Fixed Windows to Length Generalization

Self-attention is permutation invariant without position information. The original Transformer used sinusoidal encodings [130]; GPT-style models often used learned absolute embeddings. Rotary position embedding (RoPE) injects relative-position structure by rotating query and key representations as a function of position [123]. RoPE became widely adopted because it integrates naturally with decoder attention and supports useful extrapolation properties.

However, extending a RoPE-trained model far beyond its training length is not automatic. Position interpolation rescales positions into the trained range, allowing fine-tuning to longer contexts without exposing the model to numerically unseen angles [16]. YaRN combines interpolation with frequency-dependent scaling and training heuristics for efficient context extension [97]. Industrial long-context models frequently combine such position strategies with long-sequence data and attention/kernel changes. The historical lesson is that "context length" is a joint property of positional parameterization, training distribution, attention architecture, and serving memory.

4.4 Normalization and Residual Design

Layer normalization was part of the original Transformer, but pre-norm variants became standard for stabilizing very deep language models. RMSNorm removes mean centering and normalizes by root-mean-square magnitude, reducing computation and becoming common in Llama-like architectures [147]. At very large depth, residual scaling and initialization matter because small instabilities compound. DeepNet proposed depth-scaled residual mechanisms for stable training of very deep Transformers [133]; more recent work such as Hyper-Connections explores richer residual pathways [155].

These changes are less visible than MoE but important in practice: architectural improvements that slightly reduce optimization instability can save enormous compute when a pre-training run spans thousands of accelerators. Industrial architecture is therefore partly about minimizing the probability of catastrophic training failure.

4.5 Architecture as a Joint Training–Inference Cost Surface

A useful way to summarize architectural evolution is that each component is now optimized against at least two regimes. During pre-training, throughput and stable optimization dominate. During serving, batchability, memory footprint, KV-cache growth, and latency dominate. GQA and MLA may be modest changes to the forward pass but large changes

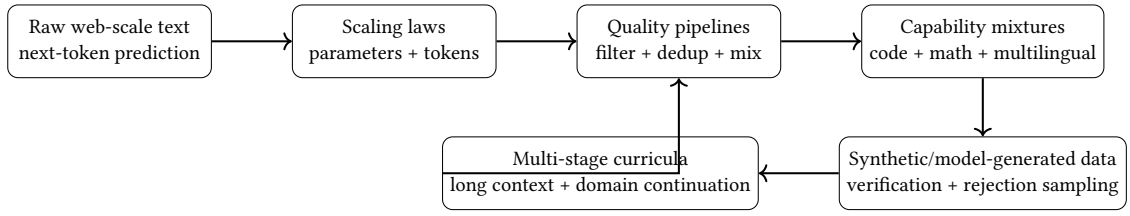


Fig. 8. Pre-training evolved from raw scale toward explicit co-design of data quality, mixture, curriculum, and synthetic generation under a fixed compute budget.

to serving economics. MoE greatly increases total parameters while controlling active compute but adds communication complexity. Sparse or linear attention improves long-sequence scaling but can complicate kernels and quality tuning.

Thus, the "best" architecture cannot be specified by loss at a fixed parameter count. It lies on a multi-dimensional Pareto frontier over training FLOPs, active inference FLOPs, parameter memory, KV memory, communication, context length, numerical stability, and quality. This systems-aware interpretation recurs throughout the rest of the survey.

Information-systems synthesis. Architecture directly changes the economics and quality frontier of information access. GQA and MLA reduce the state cost of long conversational and retrieval-augmented sessions; efficient attention determines how many retrieved documents, user-history items, or repository files can be jointly processed; and MoE increases semantic capacity without proportional active compute. These are therefore not only language-model optimizations. They determine whether conversational search, recommendation, enterprise knowledge access, and agent memory can operate under practical latency, throughput, and memory budgets.

5 Pre-Training: From Raw Scale to Data–Compute–Objective Co-Design

Pre-training supplies most of an LLM's general linguistic, factual, and procedural knowledge. Its evolution is sometimes summarized as "more tokens," but that description misses three important changes: the field learned to allocate compute more efficiently between parameters and data; data pipelines became sophisticated learning algorithms in their own right; and industrial pre-training increasingly mixes text, code, mathematics, synthetic data, multilingual corpora, and multimodal streams according to target capabilities. Figure 8 and Table 8 summarize this transition.

5.1 Objective Evolution: Why Causal Language Modeling Persisted

Early large-scale transfer learning explored multiple pre-training objectives. GPT used left-to-right autoregressive prediction [106], while BERT-style masked modeling and T5-style span corruption targeted bidirectional representation learning. For general-purpose generative systems, causal next-token prediction became the dominant backbone objective because it directly matches generation, scales cleanly, and turns arbitrary text into supervision without task labeling. The objective is deceptively simple:

$$\mathcal{L}_{\mathrm{LM}}(\theta) = -\sum_{t=1}^{T} \log p_\theta(x_t \mid x_{<t}). \tag{1}$$

Its power comes from the diversity of conditional-prediction problems implicitly contained in natural data: factual recall, syntax, translation, coding, question answering, and latent multi-step reasoning.

The persistence of causal LM training does not imply objective stagnation. Industrial recipes add domain-balanced sampling, curriculum schedules, code infilling, multi-token prediction, multimodal token prediction, and specialized

Table 8. Evolution of pre-training priorities.

Stage	Representative development	Main insight	New bottleneck created
2018–2020	GPT-style causal pre-training; GPT-3 [12] scaling	Generic next-token prediction can transfer broadly and support in-context learning.	Compute and parameter memory.
2020–2022	Scaling laws; Chinchilla [44]	Parameter count must be balanced with token count under a compute budget.	High-quality token supply and data pipeline quality.
2022–2024	Llama [128], Qwen [8], and DeepSeek [24]: multilingual, code, and math mixtures	Data composition determines capability profile; quality-weighted tokens matter.	Filtering, deduplication, provenance and contamination.
2024–2026	FineWeb-style curation; synthetic data; long-context and multimodal continuation	Data is an engineered process; curricula and model-generated data become first-class.	Feedback loops, synthetic collapse, verifier quality, and governance.

long-context phases. DeepSeek-V3, for example, augments standard next-token learning with a multi-token prediction objective designed to improve representations and potentially facilitate faster decoding [26]. The trend is therefore toward a stable universal core objective with auxiliary objectives chosen for downstream capability or systems value.

5.2 Scaling Laws: From "Bigger Is Better" to Compute-Optimal Allocation

GPT-3 crystallized the intuition that model scale can yield qualitatively new in-context behavior [12]. Kaplan et al. showed that language-model loss follows approximate power laws in model size, dataset size, and compute, encouraging predictable scaling [56]. Yet early frontier models often increased parameters faster than data. Chinchilla revised the practical optimum by empirically showing that, under a fixed compute budget, model size and training tokens should grow together; its 70B model trained on substantially more data outperformed much larger models at comparable training compute [44].

The consequence for later open and industrial models was profound. Llama 1 explicitly emphasized compute-efficient scaling and trained relatively small models on far more tokens than earlier parameter-centric recipes [128]. Llama 3 further expanded pre-training data into the many-trillion-token regime [31]. Qwen, DeepSeek, and other families similarly emphasize token scale, code/math composition, and multilingual coverage rather than raw parameter count alone [24, 26, 102]. In other words, parameter count stopped being a sufficient summary of pre-training investment.

5.3 Data Curation Becomes an Algorithmic Component

Web-scale data is not a neutral substrate. Crawls contain duplicates, boilerplate, spam, machine-generated text, low-quality SEO pages, private information, unsafe content, and benchmark leakage. As models grew, the marginal value of adding raw tokens decreased while the value of filtering, deduplication, and composition increased. Modern data pipelines typically include language identification, document-quality scoring, rule-based filters, model-based classifiers, exact and approximate deduplication, personally identifiable information removal, contamination checks, and mixture reweighting.

Open data research makes this visible. FineWeb documents the construction of a 15-trillion-token corpus and shows that filtering and deduplication decisions materially affect downstream quality; FineWeb-Edu further demonstrates that selecting educational text can improve reasoning- and knowledge-heavy benchmarks [96]. This supports a general

principle: *effective data scale is quality-weighted scale*. A token from a carefully selected source can contribute more learning signal than many low-information or duplicated tokens.

Deduplication has two roles. First, it improves statistical efficiency by reducing repeated content. Second, it reduces memorization and benchmark contamination. The two roles interact with evaluation: an apparent benchmark gain is scientifically weaker if training/test overlap cannot be excluded. As models increasingly train on synthetic and generated data, provenance tracking becomes even more important.

5.4 Data Composition: Language, Code, Mathematics, and Domain Mixtures

General web text is insufficient for frontier capability. Code teaches formal structure, long-range dependencies, APIs, and executable semantics; mathematics provides dense verifiable reasoning; multilingual corpora broaden transfer and user coverage; scientific documents add specialized knowledge; and conversational/instruction-like material changes style and controllability. Industrial pre-training therefore optimizes a mixture rather than a monolithic corpus.

This mixture is not static. Early training may emphasize broad coverage, while later stages upweight high-quality code, mathematics, or long documents. Some models use staged curricula that increase maximum sequence length or domain difficulty. The exact recipe is often proprietary, but cross-family reports consistently indicate that composition is a major source of differentiation. Qwen2.5 reports substantial data expansion and specialized coder/math variants [50, 101, 102]; Llama 3 describes extensive filtering and data-quality pipelines [31]; DeepSeek models emphasize code and mathematical capability as part of their training ecosystem [24, 115].

5.5 Tokenization: A Small Component with System-Wide Effects

Subword tokenization determines sequence length, multilingual efficiency, code granularity, and ultimately training and serving cost. Byte-pair encoding (BPE) [113] and SentencePiece [63] became dominant because they balance vocabulary size and open-vocabulary coverage. Modern tokenizers often use vocabularies much larger than early GPT systems and are explicitly optimized for multilingual scripts and code.

A tokenizer that represents Chinese, code, or structured text inefficiently consumes more context and compute for the same semantic content. Conversely, increasing vocabulary size enlarges embeddings and output projections. Tokenizer design is therefore a hidden compute-allocation decision. This becomes more important in multimodal systems, where text tokens compete with image, audio, or video tokens for a shared context budget.

5.6 Optimization: AdamW, Stability, and New Large-Scale Optimizers

Adam and AdamW remain the default optimizers for LLM pre-training because adaptive moments stabilize large-scale stochastic optimization [62, 76]. As scale increases, however, optimizer-state memory and stability become major costs. Low-precision optimizer states, distributed optimizer sharding, gradient clipping, carefully tuned warmup/decay, and loss-spike monitoring are now standard engineering components.

Recent work also revisits optimizer geometry. Muon applies matrix-aware orthogonalization-like updates to two-dimensional parameters and has attracted attention as an alternative to Adam-like optimization for large models [55]. Kimi K2 reports a scaled variant, MuonClip, as part of stable trillion-parameter MoE training [60]. Whether such optimizers become universal remains open, but the trend is notable: once architecture and data mature, optimizer efficiency again becomes a frontier variable.

5.7 Synthetic Data and the Model-in-the-Loop Era

Synthetic data first became prominent in instruction tuning, but it increasingly influences pre-training and capability-specific continued training. Strong models can generate explanations, code, translations, problem variations, critiques, or filtered rewrites at a scale that human annotation cannot match. This creates a self-improvement loop: models help construct the next model's data.

The benefit is controllable coverage; the risk is distribution collapse, amplified errors, stylistic homogenization, or hidden leakage from the teacher. High-quality synthetic pipelines therefore use verifiers, execution, rejection sampling, diversity controls, and mixtures with human or naturally occurring data. The important historical transition is from *collecting data* to *engineering data-generating processes*.

5.8 Pre-Training Is Increasingly Multi-Stage

Contemporary pre-training is better modeled as a sequence of phases than one homogeneous run. A plausible generic pipeline is broad web-scale training, followed by quality upweighting, code/math specialization, long-context extension, and possibly multimodal continuation. The learning rate, data mixture, context length, precision regime, and even objectives may change between phases. This reduces the usefulness of simple statements such as "trained on N tokens": two models with the same token count can experience very different effective curricula.

The long-term trend is clear. Pre-training evolved from maximizing raw token throughput to co-designing parameter count, token count, data quality, domain composition, sequence length, and optimization stability under finite compute. This makes pre-training one component of a larger system whose downstream capability also depends on retrieval, memory, post-training, tools, and serving design.

Information-systems synthesis. Pre-training defines the semantic substrate on which retrieval, ranking, recommendation, query understanding, and domain-specific information access depend. Multilingual and domain mixtures affect which users and collections are represented; code and mathematical data improve structured reasoning over technical information; and curation, deduplication, and provenance influence both factual reliability and downstream retrieval contamination. For information systems, pre-training data should therefore be viewed not merely as a source of language fluency but as a design choice that shapes coverage, bias, freshness, and semantic interoperability across information domains.

6 Post-Training: Alignment, Reasoning, Long Context, and Memory

Pre-training creates broad capability but does not directly optimize helpful assistant behavior, refusal boundaries, output format, tool conventions, deliberate reasoning, or robust use of extended context. We use *post-training* in a broad lifecycle sense: supervised and preference alignment, reasoning-oriented reinforcement learning, and continued/extension training that teaches a pre-trained model to operate reliably over longer contexts and memory hierarchies. Over roughly four years, the dominant recipe moved from supervised instruction tuning (SFT), to preference-based RLHF and direct preference optimization, to reinforcement learning on verifiable outcomes and trajectories, while long-context adaptation evolved from positional extension toward data-, memory-, and system-aware training. Figure 9 and Table 9 summarize the alignment and reasoning sequence, while Figure 10 presents the broader layered pipeline in a more survey-style form.

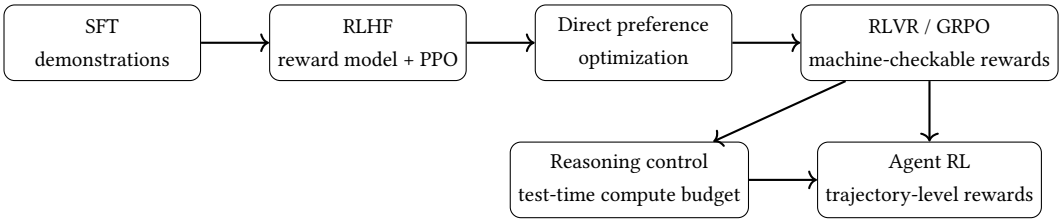


Fig. 9. Post-training evolved from imitating demonstrations to optimizing preferences, verifiable outcomes, and finally long-horizon trajectories.

Table 9. Evolution of post-training objectives and feedback.

Method	Feedback source	Strength	Limitation / next bottleneck
SFT	Human or synthetic demonstrations	Stable, simple, teaches interaction protocol and formats	Imitates known behavior; expensive to cover long-tail tasks.
RLHF/PPO	Pairwise human preferences via reward model	Optimizes subjective helpfulness/safety beyond likelihood	Complex online RL; reward hacking and label bias.
DPO-family	Offline chosen/rejected pairs	Simpler infrastructure; no explicit reward-model PPO loop	Limited exploration beyond static preference data.
Process/outcome rewards	Step labels or final correctness	Better credit assignment or scalable objective feedback	Process labels costly; outcomes sparse.
GRPO/RLVR	Verifiers, tests, exact answers	Scales reasoning RL in math/code and other checkable domains	Coverage limited by verifier availability and correctness.
Agent RL	Environment/task success	Optimizes long-horizon action sequences directly	Sparse rewards, asynchronous environments, state and safety complexity.

6.1 Supervised Fine-Tuning: Teaching the Interaction Protocol

SFT trains a pre-trained model on demonstrations (x, y) with the same token-level likelihood objective but a task- or conversation-specific distribution. Its role is often misunderstood. SFT does not need to inject all world knowledge; instead, it teaches the model how to expose latent capability through instructions, dialogue roles, formatting, refusal patterns, and task conventions. InstructGPT used a relatively small set of human demonstrations before preference-based RL [93].

Self-Instruct showed that a capable model can bootstrap instruction data by generating tasks and answers, filtering them, and fine-tuning on the resulting synthetic corpus [135]. LIMA pushed the "quality over quantity" argument further: a 65B Llama model fine-tuned on only 1,000 curated examples displayed strong instruction-following behavior, suggesting that much knowledge is learned during pre-training and that alignment data primarily teaches behavior [153]. Later instruction-tuning pipelines nevertheless expanded greatly because the target behavior itself broadened to coding, multilingual conversation, multimodality, safety, tool use, and complex reasoning.

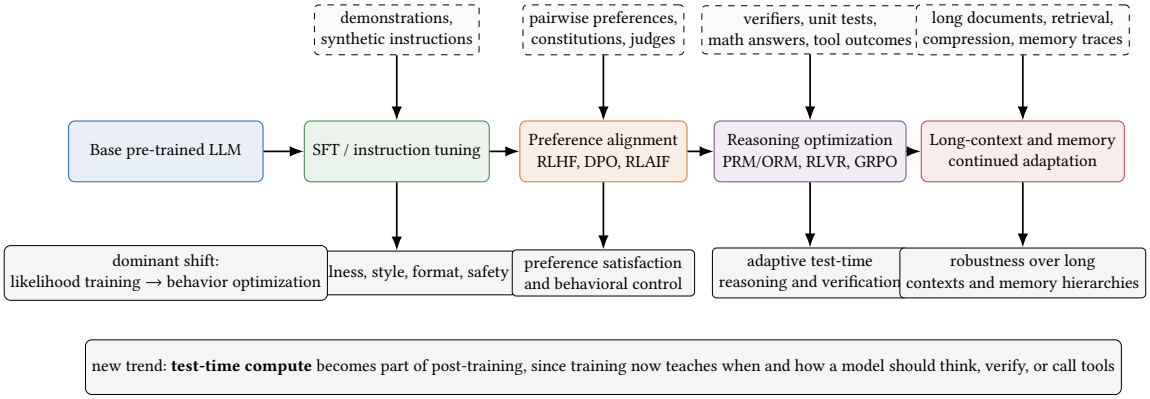


Fig. 10. A survey-style post-training stack. Inspired by recent surveys of post-training and multimodal LLMs, the figure emphasizes that post-training is no longer a single alignment step: it is a layered pipeline that progressively shapes interaction behavior, preferences, reasoning, and long-context robustness.

Parameter-efficient methods lowered the cost of adaptation. LoRA constrains weight updates to low-rank matrices [47]; QLoRA combines low-rank adaptation with a frozen quantized base model to make large-model fine-tuning possible on much smaller hardware [30]. These methods are especially important for the open ecosystem, where organizations and users specialize a shared foundation model without reproducing full training.

6.2 RLHF: Preferences Become a Training Signal

RLHF adds an explicit preference signal. The canonical pipeline collects pairwise rankings of model responses, trains a reward model, and optimizes the policy with reinforcement learning while regularizing against a reference model. InstructGPT operationalized this recipe at scale using PPO [93]. Earlier work on learning from human preferences [19] and summarization from human feedback [122] provided the methodological foundation.

The key conceptual change is that the target is no longer "what text is likely?" but "what output is preferred under a behavioral objective?" This makes post-training a control layer over the pre-trained distribution. It also creates new failure modes: reward hacking, annotator bias, mode collapse, over-refusal, and preference drift. The quality of the reward model becomes a bottleneck, and online RL requires expensive generation during training.

Constitutional AI and related RLAIF approaches replace or augment human judgments with AI-generated critiques and principle-guided preferences [11]. This is an early example of model-generated supervision entering alignment loops, foreshadowing the broader use of synthetic data and automated verifiers.

6.3 Direct Preference Optimization and the Simplification of Alignment

DPO showed that, under a particular preference-modeling formulation, one can optimize a policy directly from chosen/rejected response pairs without separately fitting a reward model and running PPO [108]. The method greatly simplified infrastructure and became a common baseline for open-model alignment. Its success illustrates an important evolutionary pattern: once a complex industrial pipeline proves value, research often finds simpler objectives that recover much of the effect.

Direct/offline preference methods are attractive because they avoid unstable online RL and can reuse static datasets. Their limitation is equally important: they cannot explore beyond the support of the collected preferences. When the

objective is to discover new reasoning strategies, interaction policies, or long-horizon behaviors, online sampling and environment feedback regain value.

6.4 Reasoning and Test-Time Computation

6.4.1 Chain-of-Thought: Inference Becomes a Computation Surface. Chain-of-thought (CoT) prompting showed that large models can improve arithmetic and symbolic reasoning by generating intermediate steps [137]. Self-consistency improved reliability by sampling multiple reasoning paths and selecting a consensus answer [134]; Tree of Thoughts generalized the idea to explicit search over intermediate states [142]. These techniques revealed a new scaling axis: additional tokens at inference time can substitute for some parameter scaling on difficult tasks.

Initially, reasoning traces were largely elicited by prompting or supervised demonstrations. The next step was to *train for reasoning behavior*. Process-reward modeling evaluates intermediate steps [70]; outcome-reward training judges only final correctness. Verifiable domains such as mathematics and code are particularly powerful because rewards can be computed automatically by exact answers, tests, or compilers.

6.4.2 GRPO, RLVR, and the Reasoning-RL Wave. DeepSeekMath introduced Group Relative Policy Optimization (GRPO), which estimates advantages from groups of sampled responses without requiring a separate critic network in the PPO style [115]. The method reduced memory and complexity while fitting naturally with multiple sampled reasoning attempts. DeepSeek-R1 then demonstrated that large-scale reinforcement learning with verifiable rewards can incentivize extended reasoning patterns such as self-reflection, backtracking, and strategy adjustment [28]. The significance is not that SFT becomes unnecessary everywhere, but that complex reasoning can be *discovered and reinforced* rather than only imitated from human-written chains.

Kimi k1.5 similarly emphasizes scaling RL, long context, and long chain-of-thought, including multimodal reasoning [59]. Qwen3 exposes thinking and non-thinking modes in a unified model, illustrating how reasoning becomes a controllable inference policy rather than a separate product category [141]. Closed systems such as OpenAI's o-series and later routed GPT systems likewise make reasoning effort a visible part of product behavior even when training details are not disclosed [91, 92].

The broader family of reinforcement learning from verifiable rewards (RLVR) changes the economics of supervision. Human preference labels are expensive and subjective; verifiers can generate effectively unlimited reward labels for domains where correctness is machine-checkable. The limitation is domain coverage. Open-ended writing, strategy, social judgment, and many real-world agent tasks do not have perfect verifiers. Frontier post-training therefore mixes verifiable rewards with learned judges, human feedback, tool outcomes, and safety constraints.

6.4.3 Reasoning-Length Control and Adaptive Test-Time Compute. Longer reasoning is not monotonically better. Models can overthink easy tasks, enter loops, or spend tokens on low-value exploration. Test-time scaling research shows that additional sampling, search, or sequential thought can outperform simply scaling parameters under some conditions, but only if compute is allocated intelligently [121]. This turns reasoning into a metareasoning problem: the system should estimate whether another step is worth its latency and cost.

Industrial interfaces increasingly expose coarse "reasoning effort" or "thinking budget" settings. A more mature design would allocate effort dynamically based on uncertainty, verifier disagreement, task type, time limits, or expected value. Reasoning control thus connects model post-training directly to serving schedulers and product economics.

6.4.4 From Response-Level RL to Trajectory-Level RL. The next post-training frontier extends the horizon from a response to a trajectory of observations and actions. A coding agent may inspect files, search a repository, edit code, run tests, read failures, and revise. A web agent may browse, fill forms, and call APIs. Reward may arrive only at task completion. Kimi K2, GLM-5, and MiniMax-M2 describe post-training or RL infrastructure oriented toward agentic trajectories rather than isolated turns [40, 60, 82].

This transition changes both algorithm and infrastructure. Environments are asynchronous, trajectories have variable length, tools can fail, and state must persist. Experience generation can become the dominant training cost. Consequently, post-training infrastructure begins to resemble distributed simulation or systems orchestration as much as conventional supervised learning.

6.4.5 A General Evolutionary Pattern. The history of post-training can be summarized as a sequence of increasingly rich feedback channels: demonstrations teach *what to do*; preferences teach *which response is better*; verifiers teach *whether a result is correct*; process rewards teach *which intermediate steps are useful*; and environments teach *whether a sequence of actions accomplished a goal.* Each richer feedback channel expands capability while increasing infrastructure complexity and raising new alignment questions.

6.5 Long Context and Memory: Continued Training Beyond the Base Window

Long-context capability spans architecture, data, post-training, and serving rather than belonging to one stage alone. We place it inside the post-training lifecycle because many practical extensions start from an already pre-trained base model and use positional adaptation, continued training on longer sequences, retrieval-oriented supervision, and memory-aware instruction tuning to make the enlarged window usable. Architectural and systems mechanisms are cross-referenced where they are introduced.

Context length expanded from a few thousand tokens in early GPT systems to hundreds of thousands or millions in recent models. This growth is often advertised as a scalar specification, but effective long-context capability depends on at least five components: positional encoding, attention computation, KV-cache state, training data at long lengths, and the model's ability to retrieve and integrate information. Table 10 summarizes the main stages.

Table 10. Long-context evolution and the bottleneck addressed by each technique.

Technique	Mechanism	Primary bottleneck	Key caveat
RoPE / position scaling	Relative rotational encoding; interpolation or frequency rescaling	Position extrapolation	Does not by itself teach models to use distant evidence.
FlashAttention	IO-aware exact attention kernels	HBM traffic and attention throughput	Asymptotic attention remains quadratic.
Sparse/block/hybrid attention	Restrict or approximate token interactions	Quadratic sequence compute	Quality depends on routing/pattern preserving relevant dependencies.
MQA/GQA/MLA	Share or compress K/V state	KV-cache memory/bandwidth	Architectural trade-offs and implementation complexity.
Paged/prefix KV caching	Virtualized cache blocks and reuse	Fragmentation and repeated prefix computation	Cache residency and eviction become scheduler problems.
RAG/external memory	Retrieve selected external evidence	Finite context and stale knowledge	Retrieval quality and evidence selection can dominate.
Long-context training	Expose model to long documents/tasks	Effective use of distant information	Expensive data and training; claimed window may exceed effective window.

6.5.1 The Quadratic Attention Barrier. Dense self-attention computes all token-token interactions, producing $O(n^2)$ attention-score work for sequence length n. Training long sequences therefore increases compute and activation memory sharply. IO-aware kernels such as FlashAttention reduce memory traffic without changing exact attention [21, 22], which pushed the practical frontier substantially. Yet exact kernels do not eliminate quadratic scaling.

Sparse and hybrid methods change the interaction pattern. Block-sparse attention limits each query to selected regions; MoBA learns or computes routing over blocks [77]; Kimi Linear and Lightning Attention target more favorable long-sequence scaling [61, 80, 100]. The design challenge is preserving the rare long-distance dependencies that make long context useful while avoiding full pairwise cost.

6.5.2 Position Extension Is Necessary but Not Sufficient. RoPE-based models can be extended with interpolation or frequency rescaling [16, 97], but increasing the maximum position index does not guarantee robust understanding. Models trained primarily on short documents may learn to ignore distant evidence or suffer position bias. Long-context continued training therefore exposes models to long documents, multi-document mixtures, and synthetic retrieval tasks so that the architecture learns to use its enlarged window.

Gemini 1.5 demonstrated strong industrial interest in million-token multimodal contexts [36]. Llama, Qwen, Kimi, DeepSeek, and other families have also extended context windows through combinations of RoPE scaling, attention changes, and long-data phases. The techniques differ, but they converge on the idea that context length is a training-and-system property, not just a configuration value.

6.5.3 KV Cache: The Hidden Cost of Long-Context Generation. Autoregressive decoding stores attention keys and values for previous tokens at every layer. For long contexts, this KV cache can exceed model-weight memory. MQA, GQA, and MLA reduce per-token state; quantization can compress caches; PagedAttention improves allocation and fragmentation [64]; and prefix caching reuses states shared by repeated prompts. Mooncake treats KV cache as a cluster-level resource and decouples prefill from decode to improve long-context serving [99].

This systems view changes architectural priorities. A method that slightly increases training cost but halves KV-cache memory may be economically valuable in production. Conversely, a model with a nominal million-token context may be impractical at high concurrency if every request reserves enormous state.

6.5.4 Long Context Is Not Long Memory. A context window is ephemeral working memory. Once a conversation or trajectory exceeds the window, the model needs external memory, summarization, retrieval, or state compression. Moreover, models may fail to use information even when it fits. "Lost in the Middle" showed that retrieval accuracy can depend strongly on where relevant information appears in the context [74]. LongBench and RULER likewise demonstrate that claimed context windows can exceed effective context under more demanding tasks [10, 46].

This distinction motivates memory architectures above the model: retrieval-augmented generation (RAG) [66], hierarchical summaries, vector databases, structured scratchpads, and agent state stores. In long-running applications, deciding *what to remember* is as important as increasing the physical window.

6.5.5 RAG and Long Context Are Complementary. RAG retrieves a small subset of external information into the model's prompt, reducing the need to load an entire corpus [66]. Long context, in contrast, lets the model jointly reason over more evidence after retrieval. The methods solve different bottlenecks: retrieval attacks search and storage; context capacity attacks joint computation and integration.

A mature system therefore often uses both. Retrieval chooses candidate documents; a long-context model compares them; an agent decides whether more retrieval is needed; and persistent memory records useful outcomes. This stack is why we later argue that long context is becoming an "operating-system" problem involving scheduling, caching, retrieval, compression, and memory hierarchy.

Information-systems synthesis. Post-training determines how an LLM behaves once it is embedded in an information service. SFT teaches interaction conventions; preference optimization controls helpfulness and ranking of candidate responses; reasoning-oriented RL improves evidence synthesis and decision support; and long-context or memory adaptation determines whether the system can preserve state across extended information-seeking sessions. In RAG, recommendation, conversational search, and agent settings, the objective is therefore not simply to generate a plausible answer but to learn a policy for when to retrieve, what evidence to trust, how much reasoning to spend, when to ask for clarification, and when to act.

7 Multimodal LLMs: From Frozen Encoders to Native Omni-Modal Models

Multimodal LLMs extend the language-model interface to images, audio, video, and other sensor streams. Their evolution mirrors the text-only story: early systems connected strong frozen components with lightweight adapters; later systems trained larger cross-modal stacks jointly; current industrial models increasingly treat multimodality as a native property of the foundation model. Figure 11 and Table 11 highlight the temporal progression. To make the section read more like a modern survey article, Figure 12 and Table 12 additionally summarize the main design axes of multimodal LLMs.

7.1 Stage I: Connect a Pre-Trained Vision Encoder to a Language Model

A natural first approach was modular. CLIP-style contrastive encoders produced strong visual representations, while large language models provided generative reasoning. Flamingo inserted gated cross-attention layers between a frozen visual encoder and a frozen or largely frozen LM, enabling few-shot image-language tasks while reusing expensive pre-trained components [3]. BLIP-2 similarly used a lightweight querying transformer to bridge frozen image encoders

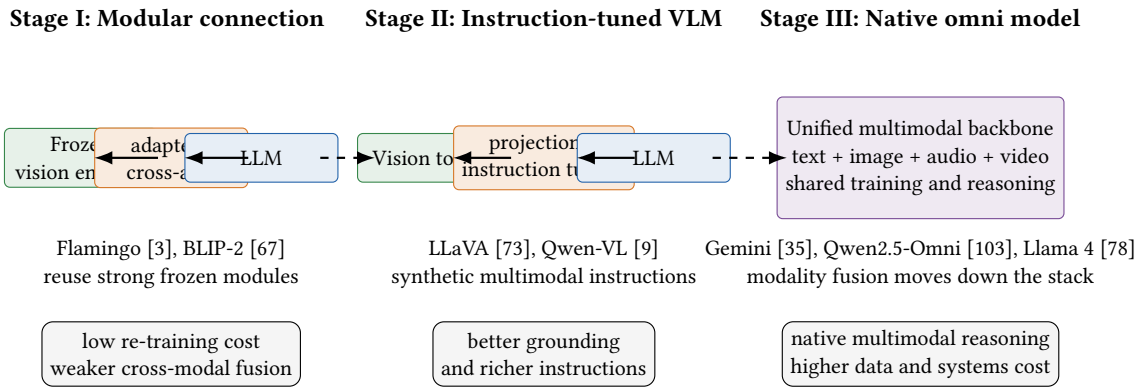


Fig. 11. A compact view of the dominant architectural progression in multimodal LLMs, following the style of recent MLLM surveys: from frozen-module composition, to vision-language instruction tuning, to jointly trained native omni-modal backbones.

Table 11. Evolution of multimodal LLM design.

Stage	Representative systems	Integration strategy	Technical implication
Frozen-component bridge	Flamingo [3], BLIP-2 [67]	Frozen vision encoder + connector/cross-attention + LLM	Cheap reuse of strong unimodal models; fusion occurs late.
Visual instruction tuning	LLaVA [73], Qwen-VL [9]	Project vision features into LLM and tune on image–instruction pairs	Synthetic instruction data makes multimodal chat accessible.
High-resolution grounding	Qwen2.5-VL [104] and related models	Dynamic resolution, OCR/localization, richer visual tokens	Token-rate and spatial-detail trade-offs become central.
Native multimodal foundation	Gemini [35], Llama 4 [78]	Multimodal training integrated into foundation model	Cross-modal capabilities can emerge during pre-training; training mixture is more complex.
Omni-modal/real-time	Qwen2.5-Omni [103], modern audio/video systems	Text, image, audio, video and generation under one interaction stack	Context budgeting, streaming latency, and unified evaluation become first-class.
Agentic multimodality	UI/robotic/computer-use agents	Perception feeds action policy and environment loop	Model must maintain state, ground actions, and recover from observation/action errors.

and frozen LLMs. The engineering advantage is clear: multimodality can be added without re-training the full language backbone from scratch.

LLaVA simplified the interface further by projecting visual encoder features into an LLM's embedding space and applying visual instruction tuning [73]. Its influence came not only from architecture but from demonstrating that synthetic instruction data generated with a strong language model could produce useful conversational vision-language behavior. This linked multimodality to the broader synthetic-data trend.

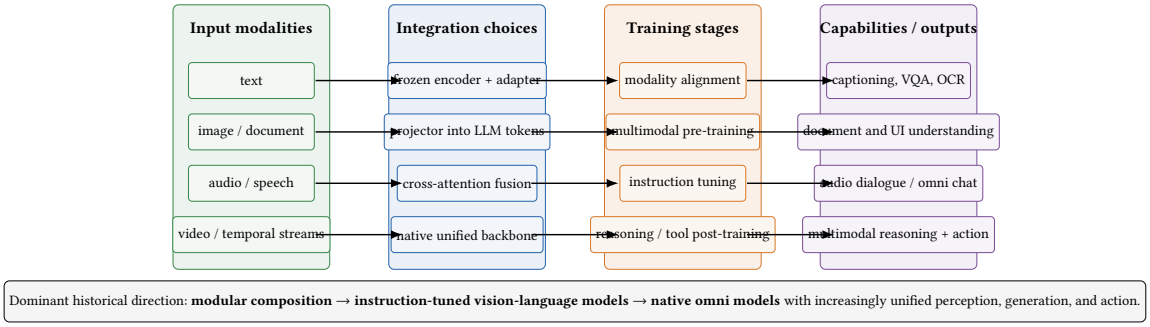


Fig. 12. A broader taxonomy of multimodal LLM design, in the spirit of recent MLLM surveys. The key axes are input modalities, integration strategy, training stage, and downstream capability. This complements Figure 11, which focuses on the temporal progression of multimodal architectures.

Table 12. Representative design patterns in multimodal LLMs. The table complements the broader MLLM surveys by summarizing the evolution from modular composition to native omni-modal systems.

Stage	Integration pattern	Training emphasis	Typical strengths	Representative examples
Modular composition	Frozen modality encoder + adapter or cross-attention into a text LLM	Keep strong unimodal modules fixed; lightweight multimodal alignment	Low retraining cost, strong reuse of existing encoders, rapid prototyping	Flamingo [3], BLIP-2 [67]
Instruction-tuned VLMs	Visual projector or connector that turns perceptual features into tokens for an instruction-tuned LLM	Synthetic multimodal instruction data and assistant-style post-training	Strong image chat, OCR, document understanding, practical deployment simplicity	LLaVA [73], early Qwen-VL [9] variants
Native multimodal backbones	Joint or tightly coupled backbone with multiple input/output modalities	Multimodal pre-training plus large-scale post-training and omni interaction tuning	Better cross-modal reasoning, unified latency path, natural extension to speech/video	Gemini [35], Qwen2.5-Omni [103], Llama 4 [78]
Agentic multimodal systems	Multimodal perception combined with tools, environment actions, and memory	Computer-use tuning, action grounding, recovery and verification	UI control, document workflows, embodied or browser agents	Computer-use assistants, coding or GUI agents

7.2 Stage II: Better Visual Tokenization and Grounding

As visual-language models became more capable, coarse image-level understanding was insufficient. OCR, charts, documents, spatial grounding, object localization, and high-resolution images require retaining fine visual detail. Qwen-VL emphasized localization and text reading [9]; later Qwen2.5-VL models improved dynamic-resolution processing and structured visual reasoning [104]. The underlying challenge is token economics: representing a high-resolution image with too many tokens consumes context and attention compute, while too few tokens destroy detail.

This led to adaptive resolution, patch merging, resampling, and modality-specific compression. The same problem appears more strongly for video: a minute of video contains many frames, and naive frame tokenization can overwhelm the language context. Efficient multimodality is therefore fundamentally a representation-rate problem.

7.3 Stage III: Native Multimodal Pre-Training

Gemini framed multimodality as a property trained into the model family rather than attached after text pre-training [35]. Gemini 1.5 combined this with long context, enabling joint reasoning over large text, image, audio, and video inputs [36]. Llama 4 similarly moved Meta's open family toward native multimodality and MoE [78]. Qwen2.5-Omni and later omni models target unified perception and generation across text, image, audio, and video [103, 105].

Native multimodality changes the representation-learning problem. Instead of mapping another modality into a language model after the fact, the model can learn cross-modal correspondences during foundation training. The potential benefit is stronger grounding and transfer; the cost is a more complex data mixture, synchronization problem, tokenizer design, and evaluation burden.

7.4 Audio and Real-Time Interaction

Audio introduces temporal constraints absent from static images. A conversational system must process streaming speech, preserve speaker and prosodic information, and generate low-latency output. Architectures may use speech encoders plus language decoders, discrete audio tokenizers, or unified continuous/discrete representations. Product systems additionally require endpointing, interruption handling, and streaming decode.

The evolution from "speech-to-text + LLM + text-to-speech" pipelines toward end-to-end or tightly integrated audio-language models is important because intermediate text can discard emotion, acoustic context, and timing. Omni-modal models increasingly attempt to reason over audio directly and produce audio outputs without a purely textual bottleneck.

7.5 Video, Temporal Reasoning, and World Modeling

Video extends image understanding with temporal order, causal dynamics, and much larger token volumes. Long-context architecture becomes directly relevant: a model may need to integrate events separated by minutes while attending to sparse critical frames. Efficient frame sampling, hierarchical visual representations, and compressed temporal tokens are therefore essential.

Multimodal reasoning also moves LLMs closer to world models. A system that sees a user interface, hears speech, reads text, and takes actions is not merely a language generator. It must maintain a state estimate over a partially observed environment. This creates a natural bridge from multimodal LLMs to agents.

7.6 Multimodal Post-Training

Instruction tuning, preference optimization, and reasoning RL all extend to multimodal data. Visual instruction tuning teaches conversational conventions; preference data can distinguish grounded from hallucinated descriptions; and verifiable multimodal tasks provide rewards for diagrams, geometry, charts, or UI interaction. Kimi k1.5 is notable for combining long-CoT reasoning with multimodal inputs [59]. As tools and agents become visual, multimodal post-training increasingly includes actions as well as answers.

7.7 The Main Trend: Modality Boundaries Move Down the Stack

The historical progression can be summarized as a downward movement of modality fusion. In adapter-based systems, modalities meet near the top of the stack. In native models, they meet during foundation training. The more fusion moves downward, the more cross-modal capability can become emergent rather than bolted on—but the more expensive

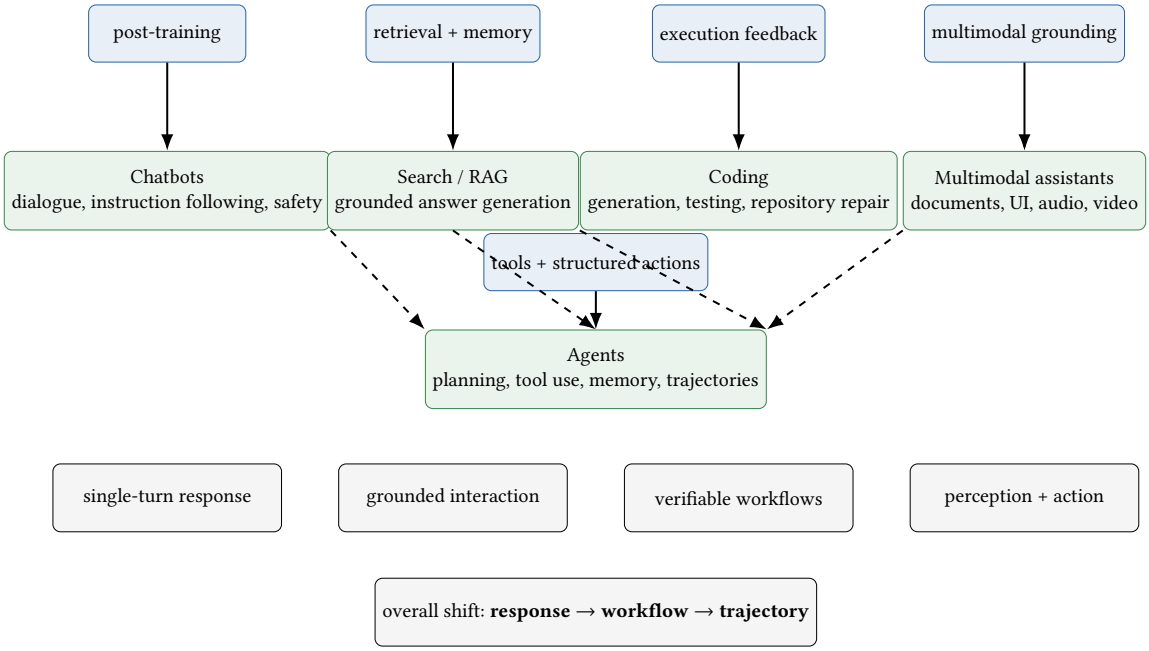


Fig. 13. Application landscape of modern LLMs. Survey-style application diagrams are useful because they expose how different products are compositions of a few recurring technical ingredients. Agents sit at the convergence point of chat, retrieval, coding, and multimodal interaction.

data, training, and systems integration becomes. This is why multimodal progress is tightly coupled to long context, efficient attention, and distributed training.

Information-systems synthesis. Multimodality broadens the very definition of an information object. Documents combine text, tables, figures, and layout; enterprise workflows include screenshots and user interfaces; search may begin from an image or spoken query; and assistants increasingly operate over audio and video streams. Native multimodal LLMs can therefore improve information systems by unifying retrieval, grounding, reasoning, and explanation across heterogeneous media. The central systems challenge shifts from text retrieval alone to cross-modal indexing, evidence alignment, provenance, and low-latency interaction.

8 Applications: From Chatbots to Tool-Using Agents

The application layer is where architectural, post-training, multimodal, retrieval, and systems advances are composed into user-facing capabilities. The dominant application pattern has changed from a single-turn text generator to an interactive assistant with retrieval, code execution, multimodal perception, persistent state, and external actions. Figure 13 and Table 13 summarize representative application classes and the technical transitions that enabled them. Figure 14 and Table 14 provide a more survey-style taxonomy that links applications to enabling ingredients, evaluation protocols, and deployment risks. Agents are treated here as an application paradigm rather than as a separate model family because agent quality depends on the underlying LLM, post-training, memory, tools, and serving stack.

Table 13. Evolution of major LLM application classes and the technical capabilities that made them practical.

Application	Visible growth phase	Main technical enablers	Current bottleneck / evaluation target
Chatbots and assistants	2022–	Instruction tuning, RLHF/DPO, dialogue formatting, safety post-training, reasoning routing	Helpfulness and personalization under latency, safety, and cost constraints; human-preference evaluation
Search / RAG assistants	2020–	Dense retrieval, RAG, browsing tools, long context, citation/provenance pipelines	Retrieval quality, evidence faithfulness, freshness, prompt injection, end-to-end answer attribution
Coding and software engineering	2021–	Code pre-training, executable verifiers, long context, repository tools, test-driven feedback	Repository-level correctness, repair/recovery, environment cost; SWE-bench-style execution evaluation
Multimodal assistants	2023–	Vision-language alignment, native multimodal pre-training, audio/video tokenization, streaming inference	Grounding, temporal reasoning, cross-modal hallucination, real-time latency
Domain / enterprise workflows	2023–	PEFT, RAG, structured outputs, private tools, validators and policy layers	Provenance, privacy, domain drift, authorization and workflow-level reliability
Agents	2023–	ReAct/tool use, function calling, memory, multimodality, environment feedback, trajectory RL	Long-horizon success, error recovery, side effects, action security and total interaction cost

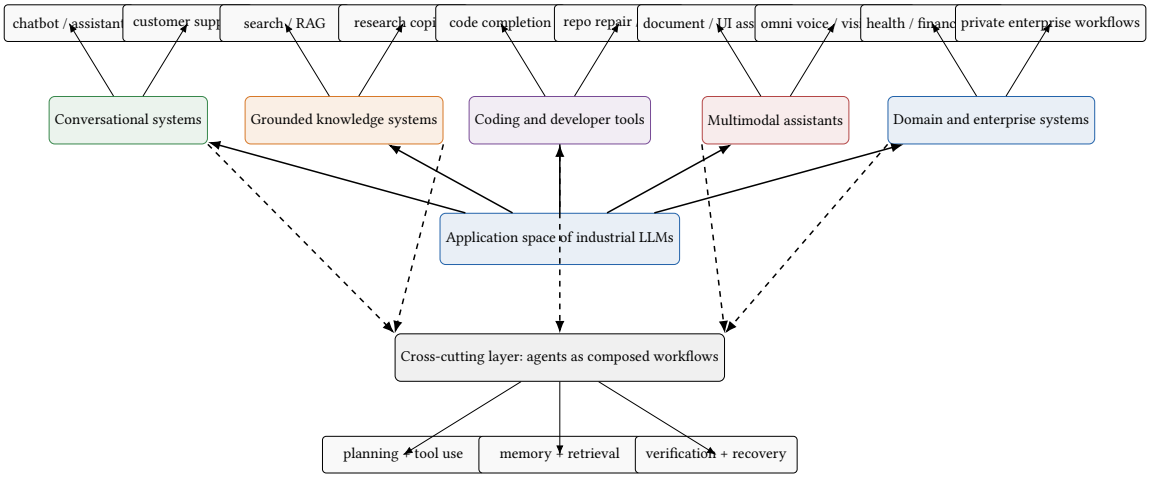


Fig. 14. A survey-style taxonomy of LLM applications. Following the presentation style of recent survey papers, the figure organizes downstream use cases into a small number of recurring classes and highlights agents as a cross-cutting composition layer rather than an isolated product silo.

8.1 Conversational Assistants and Chatbots

Chatbots were the first application that made post-training visible to a broad audience. A pre-trained language model can continue text, but a useful assistant must follow instructions, preserve dialogue state, adapt tone, decline unsafe requests, and recover when the user changes goals. Instruction tuning and RLHF transformed this interaction protocol: InstructGPT showed that comparatively small amounts of demonstration and preference data could make a base model substantially more useful to humans [93]. Later preference optimization simplified this pipeline, while chat-oriented evaluation shifted toward pairwise human judgments and arena-style comparisons [17, 108, 151].

The technical bottleneck in chat has therefore moved from fluency to control. Modern assistants must route between direct answering, long reasoning, retrieval, tools, and multimodal interaction while maintaining latency and cost targets. Conversation quality is also increasingly a memory problem: short-term dialogue fits in the context window, whereas long-lived personalization or task continuity requires retrieval, summarization, or structured external state rather than unbounded prompt growth.

Table 14. Applications as compositions of technical ingredients. Survey papers are often most useful when they connect downstream applications back to the enabling methods, evaluation protocols, and deployment risks.

Application class	Core technical ingredients	Typical evaluation	Dominant bottlenecks	Common risks / failure modes
Chatbots and assistants	SFT, preference alignment, safety tuning, retrieval-optional memory	human preference, helpfulness/safety, dialogue benchmarks	over-refusal, hallucination, persona drift	unsafe advice, style inconsistency, stale knowledge
Search / RAG / knowledge assistants	retrieval, re-ranking, grounding prompts, citation formatting, long context	grounding accuracy, citation precision, answer faithfulness	retrieval quality, context packing, latency	prompt injection, unsupported claims, citation mismatch
Coding and software engineering	code pre-training, execution feedback, repository retrieval, tool use, verifiers	HumanEval, MBPP, SWE-bench, pass@k, test success	environment setup, long trajectories, tool reliability	compilation failures, shallow fixes, brittle patching
Multimodal assistants	encoders/projectors, multimodal post-training, OCR/UI perception, speech or video handling	MMMU, DocVQA, ChartQA, instruction-following, latency	visual grounding, multimodal data scale, streaming cost	perception errors, grounding mistakes, slow interaction
Domain / enterprise systems	RAG, privacy controls, adapters, structured output, workflow integration	task-specific accuracy, compliance metrics, human audit	data governance, authorization, custom schemas	leakage, policy violation, hidden distribution shift
Agents	planning, function calling, memory, tool execution, trajectory-level RL or imitation	task completion, recovery rate, action efficiency, benchmark environments	asynchronous environments, credit assignment, observation overload	tool misuse, cascading failures, unsafe actions

8.2 Information Access: Search, RAG, Generative Retrieval, and Recommendation

Information access is one of the clearest interfaces between LLMs and the traditional concerns of information systems. Before RAG became a product pattern, dense retrieval had already shifted open-domain search from sparse lexical matching toward learned semantic representations. DPR uses dual encoders for scalable passage retrieval [57], while ColBERT preserves token-level late interaction to improve ranking effectiveness without running full cross-attention over every candidate [58]. These systems established a modular architecture in which retrieval and generation can evolve independently.

Retrieval-augmented generation (RAG) connects a language model to an external corpus and conditions generation on retrieved evidence [66]. Atlas showed that retrieval-augmented pre-training can improve few-shot knowledge-intensive learning while keeping the external index updateable [51]. WebGPT further demonstrated that browsing and evidence gathering can be integrated with human-feedback training [85]. The application consequently evolved from "retrieve once, then answer" toward iterative search in which the model reformulates queries, compares sources, requests

Table 15. Evolution of LLM-based information access, a key bridge between general LLM research and intelligent information-systems research.

Stage	Retrieval / ranking paradigm	Role of the LLM	Evaluation emphasis	Representative work
Dense retrieval	Dual-encoder or late-interaction retrieval	Encodes query/document semantics; generator may be separate	Recall@k, MRR, nDCG, QA accuracy	DPR, ColBERT
RAG	Retriever + generator over retrieved evidence	Conditions answers on external, updateable knowledge	answer accuracy, factuality, citation quality, retrieval recall	RAG, Atlas
Generative retrieval	Model maps queries directly to document identifiers	Retrieval itself becomes sequence generation / model memory	retrieval accuracy, generalization to corpus scale	DSI
Adaptive RAG	Model decides whether/when to retrieve and critiques evidence	Retrieval becomes an inference-time policy	retrieval utility, support/faithfulness, answer quality, cost	Self-RAG
LLM ranking / recommendation	Candidate generation followed by LLM reranking or generative recommendation	Uses semantic/world knowledge to rank or explain items	nDCG/MRR, ranking bias, calibration, personalization	LLMRank; LLM recommendation surveys
Agentic search	Iterative query reformulation, browsing, evidence accumulation, tool use	Controls a multi-step information-seeking trajectory	task success, evidence coverage, citation precision, latency/cost	WebGPT [85]-style browsing and retrieval agents

additional evidence, and decides when the evidence is sufficient. Self-RAG makes that policy explicit by teaching a model when to retrieve and how to critique retrieved evidence and its own output [6].

A complementary line collapses retrieval into generation. Differentiable Search Index (DSI) maps queries directly to document identifiers with a Transformer, treating model parameters themselves as an index [124]. This idea highlights a recurring design choice: knowledge can live in parametric weights, a vector or lexical index, long context, or an external tool. Each location has different update, memory, latency, and provenance properties.

Recommendation adds personalization and ranking to this picture. LLMRank demonstrates that general-purpose LLMs can act as zero-shot candidate rankers, while also exposing order, popularity, and prompt-position biases [45]. The wider LLM-for-recommendation literature distinguishes discriminative uses of language representations from generative recommendation and explanation [140]. From a systems perspective, search and recommendation increasingly share a common stack: candidate retrieval, semantic reranking, long-context user or session state, generation, and policy-level constraints.

RAG changes the failure surface rather than eliminating hallucination. Retrieval can return irrelevant, stale, conflicting, or adversarial content, and the model may ignore useful evidence even when it is present. Effective knowledge assistants therefore require retrieval evaluation, citation/provenance handling, reranking, context compression, and policies for distinguishing parametric knowledge from retrieved evidence. Long context and RAG are complementary: retrieval narrows a corpus, while long context permits joint reasoning over a larger evidence set. Table 15 summarizes this evolution from dense retrieval to agentic information seeking.

8.3 Coding and Software Engineering

Code-specialized LLMs extend language modeling into an environment with unusually strong feedback: programs can be parsed, compiled, executed, unit-tested, and reviewed through repository state. This makes coding a natural bridge from generation to verifiable reasoning and then to agents. Code pre-training and instruction tuning improve local completion and synthesis; execution feedback supplies scalable rewards; repository-level tasks require search, editing, testing, and iterative repair. SWE-bench captures this transition by evaluating whether a system can resolve real software issues in existing repositories rather than merely generate isolated functions [54].

The application has consequently evolved from autocomplete to software-engineering workflows. A capable coding system may inspect multiple files, infer project conventions, invoke a shell, run tests, read failures, and revise patches. Serving and context management become part of coding quality because repositories are much larger than a single prompt and tool outputs can dominate the token budget.

8.4 Multimodal Assistants and Interactive Interfaces

Multimodal LLMs extend assistant applications from text to documents, images, audio, video, and graphical interfaces. LLaVA showed the effectiveness of vision-language instruction tuning with a comparatively simple connector design [73]; Gemini and later native multimodal models moved toward jointly trained backbones that treat modality integration as a foundation-model property [35]. Real-time speech models further make latency, streaming, interruption handling, and turn-taking central parts of the application design.

This application class is important because it collapses boundaries among perception, reasoning, and action. Reading a chart, interpreting a screenshot, discussing a document, or controlling a graphical interface all require the model to ground language in non-text observations. The same capability becomes a prerequisite for computer-use agents described below.

8.5 Domain and Enterprise Applications

Enterprise and scientific deployments often prioritize controllability, provenance, privacy, and domain accuracy over open-ended conversational breadth. Parameter-efficient adaptation such as LoRA and QLoRA lowers the cost of domain specialization [30, 47], while RAG allows frequently changing knowledge to remain external to model weights. In practice, many domain systems combine a general foundation model with organization-specific retrieval, constrained output formats, validation rules, and tool APIs.

The trend is from model customization toward system customization. Rather than fine-tuning a separate full model for every workflow, applications increasingly allocate domain knowledge among prompts, retrieval stores, lightweight adapters, tools, and validators. This modularity improves update speed but makes evaluation more end-to-end: the relevant question is whether the complete workflow is reliable under realistic data and authorization constraints.

8.6 Agents: From Tool Calls to Long-Horizon Trajectories

The agent paradigm changes the application from mapping one context to one response into a loop that repeatedly observes, reasons, invokes tools, modifies external state, and incorporates new observations. This turns language-model use into a sequential decision problem and makes quality inseparable from tool interfaces, memory, planning, environment feedback, and recovery from failure. Figure 15 and Table 16 summarize this progression.

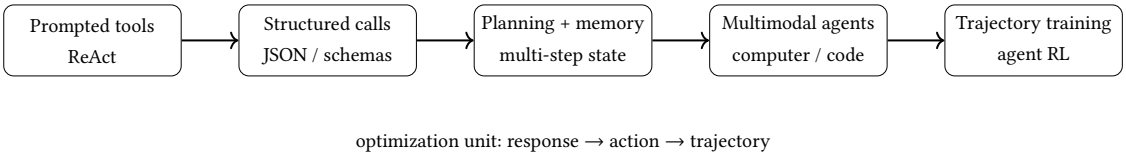


Fig. 15. Agent evolution shifts the optimization unit from a single generated response toward a long-horizon trajectory with state, tools, observations, and recovery.

Table 16. Evolution of LLM agents.

Stage	Representative idea	What is learned/controlled	Main bottleneck
Prompted tools	ReAct, Toolformer	When to reason and invoke an external capability	Brittle formats; short horizons
Structured function calling	Schemas, constrained actions	Tool selection and valid arguments	Tool coverage and semantic correctness
Planning + memory	Scratchpads, retrieval, task graphs	State persistence and subgoal management	Error accumulation and context growth
Computer-use agents	Browser, desktop, coding environments	Ground perception into executable actions	UI drift, latency, observation noise
Trajectory RL	Environment/verifier reward	Long-horizon strategy and recovery	Sparse credit, asynchronous rollouts, safety

8.6.1 Stage I: Prompted Tool Use and Interleaved Reasoning. Early agentic behavior was largely elicited through prompting. Chain-of-thought showed that intermediate reasoning could improve difficult tasks [137]. ReAct then interleaved natural-language reasoning with environment actions, demonstrating a general recipe for combining internal deliberation with external observations [143]. Toolformer moved part of this capability into training by teaching a language model to decide when and how to call external APIs [112]. These methods established three durable ideas: tools can compensate for parametric limitations; action results should be returned to the model as observations; and reasoning should be conditioned on the changing environment state rather than produced once at the beginning.

At this stage, most systems were short-horizon. A search call, calculator invocation, or single code execution was enough to obtain the missing information. Prompt design therefore played a larger role than explicit agent training. The main failure modes were malformed calls, unnecessary actions, brittle tool descriptions, and compounding errors when an early observation was incorrect.

8.6.2 Stage II: Function Calling and Structured Action Spaces. Industrial LLM APIs subsequently exposed function calling, JSON schemas, and constrained decoding as first-class interfaces. The important technical shift was not merely syntactic. Structured action spaces reduce ambiguity between natural language and executable operations, make validation possible before side effects occur, and enable deterministic routing from model outputs to tools. Tool choice became a supervised capability that could be improved through instruction data containing API descriptions, arguments, and expected execution traces.

The move toward structured actions also changed evaluation. A model can produce fluent text while selecting the wrong tool or arguments; therefore agent quality requires exact or semantic comparison of tool calls, not only

answer quality. This motivated function-calling benchmarks and execution-based scoring. It also encouraged grammar-constrained generation and schema-aware decoding in inference engines.

8.6.3 Stage III: Planning, Memory, and Environment State. Longer tasks expose the limitations of one-step tool selection. Agents need to decompose goals, preserve intermediate state, revisit failed assumptions, and decide when to stop. Tree-of-Thoughts generalized chain-of-thought by exploring alternative reasoning branches [142], while self-consistency and sampling-based methods illustrated the broader value of generating multiple candidate trajectories [134]. In practical systems, explicit planners, scratchpads, task graphs, and episodic memory are often layered above the base model.

Memory is particularly important because agent horizons can exceed the model context window. An agent may keep a compact state containing previous decisions, retrieved evidence, code artifacts, browser state, and unresolved subgoals. This creates a memory hierarchy analogous to computer systems: the transformer context acts as fast working memory, retrieval stores provide larger but indirect memory, and external files or databases provide persistent state. The agent must learn or be programmed to move information among these layers.

8.6.4 Stage IV: Computer Use, Coding, and Multimodal Agents. Once LLMs became multimodal, action spaces expanded from APIs to graphical interfaces and physical or simulated environments. A computer-use agent must jointly solve perception, grounding, planning, and control: identify a visual element, infer the intended action, issue a click or keystroke, and verify that the interface changed as expected. Coding agents similarly operate over repositories rather than isolated functions, using search, editors, test runners, shells, and version-control state. SWE-bench is representative of this transition because success requires modifying real repositories and satisfying executable tests rather than generating a standalone answer.

This stage also makes latency and context management more consequential. A trajectory may contain screenshots, tool outputs, source files, test logs, and repeated prefixes. Prefix caching, selective observation compression, and retrieval become agent-enabling inference features rather than generic serving optimizations. Multimodal perception and long context therefore converge naturally in agent systems.

8.6.5 Stage V: Training on Trajectories. The newest transition is from prompting agents to *training* them. Supervised agent tuning can imitate high-quality trajectories, but demonstration coverage is limited and can teach unnecessary action sequences. Reinforcement learning can instead optimize task success, execution correctness, or verifier outputs. The challenge is credit assignment: a final success or failure may depend on a decision made dozens of actions earlier. Long trajectories are also expensive because each environment step may require model inference, network access, sandbox execution, or a human-facing application.

This motivates trajectory-level process rewards, hierarchical objectives, curriculum design, and replay of successful interactions. It also changes systems design. Agent RL workers spend substantial time waiting for asynchronous environments, so maximizing GPU utilization may require decoupling generation from environment execution and batching trajectories at different stages. In this sense, agent training is both an RL problem and a distributed-systems scheduling problem.

8.6.6 Reliability: Recovery Matters More Than First-Pass Accuracy. Static benchmarks often reward a single correct output. Agents can be useful even with imperfect local decisions if they can detect and repair mistakes. A robust agent checks preconditions, validates tool results, notices contradictions, retries with alternative strategies, and rolls back unsafe actions. This suggests that future evaluation should measure recovery rate, wasted actions, cumulative cost, and side effects in addition to terminal task success.

The reliability challenge is amplified by non-stationary environments. Websites change, APIs evolve, search results differ, and long-running tasks encounter unexpected states. The best agent policy therefore cannot be a memorized action sequence. It must interpret observations and adapt online. This is one reason environment interaction is becoming a separate capability axis from language-model benchmark accuracy.

8.6.7 Security and Alignment in Action Space. Agents can cause effects outside the conversation, so alignment expands from response safety to action safety. Prompt injection can arrive through retrieved webpages or documents; tools may expose sensitive operations; and successful completion may conflict with authorization boundaries. Secure agents need tool-level permissions, provenance tracking, sandboxing, confirmation for consequential actions, and policies that distinguish reading from modifying external state.

The historical trajectory is thus clear: agents evolved from prompted reasoning plus occasional tools, to structured function calling, to memory-rich multimodal interaction, and finally toward policies optimized over long trajectories. The corresponding unit of capability is no longer a good sentence or even a correct answer, but a reliable sequence of decisions under changing observations.

8.7 Applications as Intelligent Information Systems

The application classes above can be understood as points on a common information-system spectrum rather than unrelated products. Chatbots emphasize interaction and presentation; AI search and RAG emphasize retrieval and grounded synthesis; recommendation emphasizes personalized selection and ranking; coding assistants combine information access with transformation and executable verification; multimodal assistants broaden the information interface; and agents add action and feedback. The stronger the LLM becomes, the more value shifts from isolated generation quality toward the quality of the complete information loop—what the system retrieves, remembers, recommends, explains, executes, verifies, and records for the next interaction. This is also why classical information-system concerns such as indexing, ranking, provenance, personalization, access control, user modeling, latency, and evaluation remain essential in the LLM era.

9 Model Training and Inference: From Distributed Optimization to State-Aware Serving

The evolution of large language models is inseparable from the systems that train and serve them. Training determines which architectures and context lengths are economically feasible; inference determines which of those trained capabilities can be exposed under realistic latency, memory, throughput, and cost constraints. Treating the two as independent subsystems obscures a central fact of industrial LLM development: architectural choices such as MoE, GQA/MLA, long context, quantization, and multimodality are increasingly selected with both the training cluster and the serving fleet in mind. This section therefore surveys training and inference as a single *model–system co-design* problem.

Figure 16 gives a unified view. Figures 17 and 18, together with Tables 17 and 18, then summarize the detailed evolution on each side.

9.1 Training: From Data Parallelism to Multi-Dimensional Parallelism

The growth of LLMs was enabled by a parallel evolution in training systems. Algorithmic ideas such as MoE, long context, and multimodal pre-training are useful only if they can be mapped efficiently onto accelerators and networks. Training-system evolution can therefore be read as a sequence of responses to three constraints: model states no longer

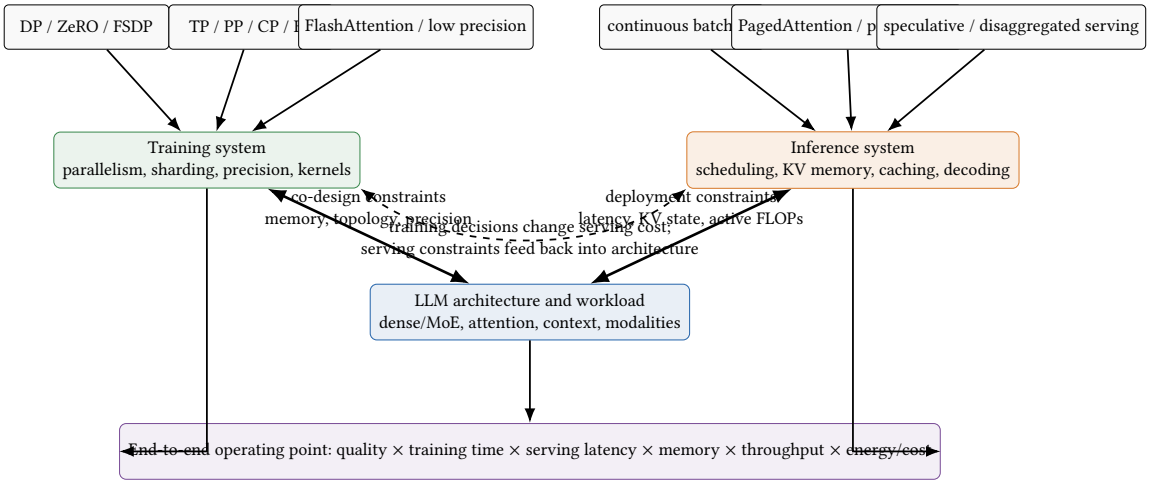


Fig. 16. Model training and inference as a single co-design problem. Modern LLM architecture is shaped simultaneously by distributed-training feasibility and deployment economics. The relevant optimization target is therefore an end-to-end operating point rather than isolated training FLOPs or tokens-per-second.

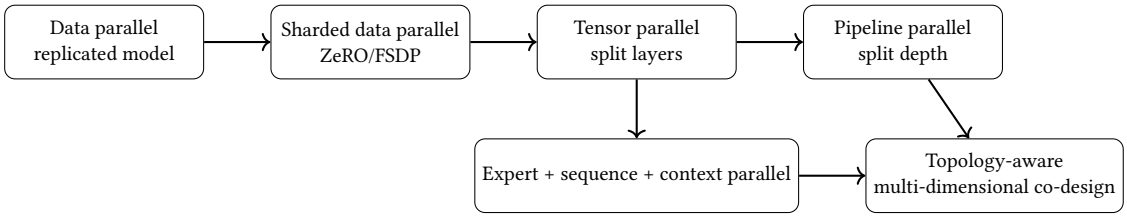


Fig. 17. Distributed LLM training evolved from replicated data parallelism toward composition of multiple parallel dimensions chosen to match model structure and cluster topology.

Table 17. Parallelism and memory techniques in LLM training.

Technique	Partitioned dimension	Primary benefit	Characteristic cost
Data parallelism	Batch/examples	Simple throughput scaling	Full model/optimizer replication
ZeRO/FSDP	Optimizer, gradients, parameters	Removes data-parallel state redundancy	Communication for gathered/sharded states
Tensor parallelism	Hidden/head/MLP dimensions	Fits and accelerates individual layers	Frequent collectives; topology sensitivity
Pipeline parallelism	Layer depth	Scales model depth and state	Pipeline bubbles and activation transfers
Expert parallelism	MoE experts	Distributes sparse capacity	All-to-all routing and load imbalance
Sequence/context parallelism	Token positions / context	Reduces activation and long-context memory	Attention communication complexity
Activation checkpointing	Stored activations	Trades memory for recomputation	Extra forward compute

fit on one device, arithmetic demand grows faster than a practical training window, and communication increasingly competes with computation. Table 17 and Figure 17 summarize the major techniques.

9.1.1 Data Parallelism: Replicate the Model, Split the Batch. Data parallelism is the simplest scaling strategy: every worker stores a full model, processes different examples, and aggregates gradients. It scales well while the model and optimizer states fit on each accelerator. For LLMs, however, optimizer states, gradients, activations, and parameters quickly exceed device memory. Mixed precision reduced this pressure, but optimizer state remained substantial, particularly for Adam/AdamW [62, 76].

ZeRO attacked this memory redundancy by partitioning optimizer states, gradients, and eventually parameters across data-parallel workers [109]. This reframed data parallelism from full replication to sharded state management. Modern frameworks expose related fully sharded data-parallel strategies in which parameters are gathered just in time for computation and re-sharded afterward.

9.1.2 Tensor Parallelism: Split Individual Matrix Operations. When a layer itself is too large or compute-intensive for one device, tensor parallelism partitions matrix multiplications across accelerators. Megatron-LM popularized transformer-specific tensor parallelism by splitting attention and MLP projections so that communication can be placed at carefully chosen boundaries [119]. The advantage is low-level scalability within a node connected by high-bandwidth links; the cost is frequent collective communication and sensitivity to topology.

Tensor parallelism strongly influenced model design. Hidden dimensions, head counts, expert counts, and vocabulary sizes are often chosen to divide cleanly across device groups. Thus, what appears in a model report as an architectural hyperparameter may partly reflect distributed-system constraints.

9.1.3 Pipeline Parallelism: Split Layers into Stages. Pipeline parallelism assigns consecutive layer ranges to different devices and streams microbatches through the stages. GPipe established a general pipeline approach for large networks [48]; Megatron-LM and related systems developed schedules that reduce idle "bubble" time and combine pipeline parallelism with tensor and data parallelism [86]. Interleaved schedules can assign multiple virtual stages per device to further improve utilization.

Pipeline parallelism makes memory scale with the number of stages but introduces scheduling complexity. Activation transfers cross stage boundaries, pipeline bubbles reduce efficiency, and the optimal microbatch count depends on memory and latency. The technique is most valuable when a model is too deep for a single tensor-parallel group.

9.1.4 3D and Multi-Dimensional Parallelism. The key systems insight of Megatron-style training is composition. Data, tensor, and pipeline parallelism address different dimensions and can be multiplied together. A training job with thousands of accelerators can form tensor-parallel groups within nodes, pipeline stages across groups, and data-parallel replicas across the remaining dimension [86]. Contemporary stacks add sequence/context parallelism for long sequences and expert parallelism for MoE layers.

This creates a topology-mapping problem. Communication-heavy tensor or expert exchanges should use the fastest links; data-parallel reductions can tolerate slower interconnects; pipeline traffic lies between. Efficient scaling therefore depends on assigning parallel dimensions to the physical cluster hierarchy rather than selecting degrees independently.

9.1.5 Sequence and Context Parallelism. Long context changes activation memory and attention communication. Sequence parallelism shards token positions for operations that do not require the full hidden state, reducing replicated activation memory. Context parallelism distributes a long sequence across devices while coordinating attention computations. These techniques matter increasingly because extending context length can consume more memory than increasing batch size.

The attention kernel itself also evolved. FlashAttention reorganizes exact attention around GPU memory hierarchy, reducing high-bandwidth-memory traffic and avoiding materialization of the full attention matrix [21, 22]. Although usually described as a kernel optimization, it has system-level consequences: longer sequences fit, recomputation patterns change, and end-to-end model FLOPs can be converted into useful throughput more effectively.

9.1.6 Expert Parallelism for MoE. MoE introduces a new distribution dimension. Experts can be placed on different devices, while routers send each token to selected experts. The resulting all-to-all communication differs from the collectives used by dense tensor parallelism. Load imbalance is also dynamic: if too many tokens choose the same expert, some devices stall while others idle. Switch Transformer and later industrial MoE systems therefore couple routing objectives with systems-aware capacity management [27, 33].

Fine-grained experts increase routing flexibility but can raise communication overhead. Shared experts reduce routing pressure but consume dense compute. DeepSeek's MoE designs are illustrative of how architectural and systems choices co-evolve: expert granularity, routed/shared capacity, auxiliary balancing, and communication strategy must be designed together [25, 26].

9.1.7 Activation Checkpointing, Precision, and Optimizer Engineering. Memory can also be traded for compute by activation checkpointing: only selected activations are stored and others are recomputed during backward. Low-precision formats reduce parameter and activation storage and increase tensor-core throughput, while maintaining numerically sensitive reductions or optimizer states at higher precision. Large-scale training increasingly mixes precision modes by operation rather than selecting a single global dtype.

Optimizer engineering has also become part of systems design. AdamW is robust but state-heavy; newer large-scale training efforts explore lower-memory optimizers or momentum variants such as Muon-like updates for matrix parameters. The broad trend is that optimizer choice is evaluated not only by convergence per step but by convergence per wall-clock dollar under memory constraints.

9.1.8 Fault Tolerance and Elasticity. At thousands of accelerators, hardware failures are expected events. Training systems therefore need checkpointing, restartable dataloaders, deterministic or recoverable optimizer state, and mechanisms to avoid losing hours of progress. Checkpoints themselves can be terabytes, making save frequency and distributed I/O performance significant. Cloud training further motivates elasticity, although changing the world size of complex 3D-parallel jobs is much harder than resizing data-parallel training.

9.1.9 The Main Trend: Parallelism Becomes Part of the Model Specification. Early papers could describe a model independently of the cluster used to train it. Frontier LLMs increasingly resist that separation. MoE depends on expert placement; long context depends on attention kernels and context parallelism; multimodality creates heterogeneous token rates; and very large dense layers are shaped by tensor-parallel divisibility. The practical "architecture" of a frontier LLM is therefore a model-system co-design that includes partitioning, precision, kernels, communication, and fault-tolerance strategy.

9.1.10 Training and Post-Training Framework Landscape. The software stack that implements these ideas is layered. Megatron-LM and Megatron Core expose GPU-optimized transformer components and multidimensional parallelism, while DeepSpeed emphasizes distributed optimization and memory efficiency through ZeRO and related techniques [29, 86, 87, 109, 119]. They are therefore closer to the execution substrate than to a high-level model-development API. By contrast, Hugging Face Transformers provides a broad model and training interface, and TRL extends that

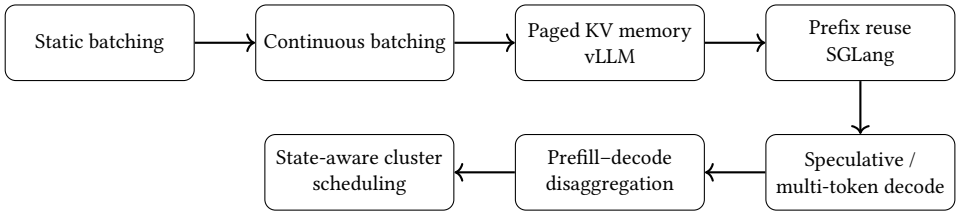


Fig. 18. LLM serving evolved from batching requests to explicitly scheduling dynamic model state, reused prefixes, speculative work, and heterogeneous prefill/decode phases.

interface with post-training algorithms including SFT, DPO, GRPO, and reward modeling [49, 139]. In practice, a TRL or Transformers job can itself use DeepSpeed underneath, illustrating that framework names often identify different abstraction layers rather than mutually exclusive choices.

SWIFT packages model adaptation, alignment, evaluation, quantization, inference, and deployment into a more integrated workflow, while Ray provides cluster-level worker and resource orchestration that can launch distributed jobs implemented in other frameworks [5, 83, 84]. For reinforcement-learning scaling, slime represents a newer class of training system: it connects Megatron-based policy training to SGLang-based rollout generation and exposes data generation, reward, verifier, and environment interaction through one training–rollout path [126]. This architecture reflects a broader shift in post-training from a single trainer process to a distributed online system whose generators, evaluators, environments, and learners run at different rates.

Table 19 compares these training frameworks together with the major inference engines discussed below. The main lesson is compositional: a production stack may use Transformers/TRL or SWIFT at the experiment layer, DeepSpeed or Megatron at the distributed-training layer, Ray for cluster orchestration, and vLLM/SGLang/TensorRT-LLM for rollout or deployment.

9.2 Inference: From Batched Decoding to State-Aware Serving

Inference became a research topic in its own right once LLM deployment cost rivaled training cost. Autoregressive generation has a distinctive systems profile: prompts are processed in a parallel *prefill* phase, while output tokens are produced sequentially in a *decode* phase. Requests have variable lengths, KV caches grow over time, and interactive workloads impose latency objectives in addition to throughput. Modern serving systems such as vLLM and SGLang therefore optimize scheduling and memory as aggressively as model kernels. Figure 18 and Table 18 show the main evolution.

9.2.1 Continuous Batching. Traditional static batching groups requests and waits until the full batch completes. This wastes accelerator capacity when sequence lengths differ. Continuous or iteration-level batching instead inserts and removes requests between decoding steps, maintaining a fuller batch and reducing head-of-line blocking. The conceptual shift is from batching complete sequences to scheduling token-generation iterations.

This scheduling model is now foundational. Most high-throughput LLM engines dynamically form batches subject to token, memory, and latency constraints. The scheduler effectively decides which requests receive the next unit of GPU time.

Table 18. Evolution of LLM inference optimization.

Technique	Target bottleneck	Core mechanism	Workloads benefiting most
Continuous batching	Variable request lengths	Rebatch between token steps	High-concurrency chat
PagedAttention	KV fragmentation/capacity	Block-virtualized KV cache	Variable-length generation, many concurrent requests
Prefix caching	Repeated prefill	Reuse KV states for shared prefixes	Multi-turn chat, agents, long system prompts
Speculative decoding	Serial decode steps	Draft multiple tokens, verify in parallel	Low/moderate entropy generation
Quantization	Weight/KV bandwidth and capacity	Lower-precision storage and kernels	Memory-bound serving and edge deployment
P/D disaggregation	Different prefill/decode resource profiles	Separate pools and transfer KV state	Long prompts and heterogeneous traffic
Architecture-aware serving	MoE/MLA/hybrid patterns	Specialized kernels, routing and cache layout	Modern sparse/long-context models

9.2.2 PagedAttention and Virtualized KV Memory. The KV cache is the dominant dynamic memory object in autoregressive serving. Naively reserving contiguous memory for each request causes fragmentation and over-provisioning because output lengths are unknown. vLLM introduced PagedAttention, organizing KV cache into fixed-size blocks and mapping logical sequence positions to non-contiguous physical pages [64]. The analogy to operating-system virtual memory is important: memory allocation becomes flexible enough to support many concurrent variable-length sequences.

Paged KV storage also enables efficient sharing. Multiple completions from the same prompt or repeated prefixes can reference common blocks rather than duplicate them. This is particularly valuable for best-of-n, beam-like sampling, system prompts, and agents that reuse long histories.

9.2.3 Prefix Caching and Structured Reuse. As prompts became longer, repeated prefill work became a major source of waste. Prefix caching stores KV states for common prefixes and reuses them across requests. SGLang generalized this idea with structured generation and radix-tree caching, allowing shared prefixes created by programs, multi-turn chat, or agent workflows to be exploited systematically [152]. The benefit can be dramatic when a long system prompt, document, or conversation prefix is reused many times.

The optimization changes application design. Prompt templates and agent frameworks can preserve stable prefixes instead of continually rewriting equivalent context. Serving performance is therefore partly determined by prompt structure, not only model size.

9.2.4 Speculative Decoding and Multi-Token Prediction. Sequential decoding underutilizes accelerators because each step performs large matrix multiplications for one new token per sequence. Speculative decoding uses a smaller draft model to propose multiple tokens, then verifies them in parallel with the target model; accepted prefixes reduce the number of expensive target-model steps [14, 65]. Medusa and related approaches attach multiple prediction heads to propose future tokens without a separate draft model [13].

These methods trade extra computation for fewer sequential synchronization points. Their speedup depends on acceptance rate, output entropy, draft quality, and batch regime. They illustrate a recurring inference principle: redundant parallel work can be cheaper than serial exact work on modern hardware.

9.2.5 Quantization and Low-Precision Serving. Weight-only and weight-activation quantization reduce memory bandwidth and allow larger models or batches to fit on a device. Inference engines increasingly support 8-bit, 4-bit, and architecture-specific low-precision formats. The engineering challenge is to realize theoretical compression as wall-clock speedup: dequantization overhead, kernel support, group size, outliers, and tensor-core compatibility all matter.

KV-cache quantization attacks a different bottleneck and becomes more valuable with long context. A serving stack may therefore use one precision for weights, another for activations, and another for cached keys/values.

9.2.6 Prefill–Decode Disaggregation. Prefill and decode stress hardware differently. Prefill processes many prompt tokens in parallel and is comparatively compute-intensive; decode repeatedly accesses weights and growing KV state for a small amount of new work and is often memory-bandwidth limited. Disaggregated serving places these phases on different workers or pools so each can be provisioned and scheduled independently. Mooncake is a prominent example for long-context workloads, treating KV cache transfer and storage as cluster-level concerns [99].

Disaggregation becomes attractive when prompts are very long, traffic is heterogeneous, or service-level objectives differ between time-to-first-token and inter-token latency. It also introduces network transfer of KV state, so topology and cache movement become part of the inference path.

9.2.7 Architecture-Aware Serving. Modern model architecture and serving systems increasingly co-design each other. MQA and GQA reduce KV memory; MLA compresses latent attention state [25]; MoE reduces active FLOPs but requires expert dispatch; hybrid or linear attention targets long-context complexity; and native multimodal models introduce visual/audio token bursts. An inference engine must understand these structures to schedule and partition efficiently.

This is why "tokens per second" cannot be compared meaningfully without workload details. Prompt length, output length, concurrency, cache hit rate, quantization, tensor-parallel degree, and model architecture all affect throughput and latency. Production evaluation increasingly reports separate prefill throughput, time to first token, inter-token latency, and request-level tail latency.

9.2.8 Serving as an Operating-System Problem. The trajectory from static batching to vLLM-style paged memory, prefix caching, speculative execution, and disaggregation resembles the evolution of operating systems. The serving layer allocates memory, schedules work, shares immutable state, moves data across devices, and enforces isolation among requests. Agentic workloads make this analogy stronger because requests can be long-lived and bursty, alternating model execution with external tool latency.

The key trend is therefore from *kernel-centric* optimization to *state-aware cluster scheduling*. Future inference performance will depend not only on faster GEMMs but on whether the system can place and reuse the right model state at the right time.

9.2.9 Inference Framework Landscape. Three serving frameworks illustrate different engineering emphases. vLLM established a general high-throughput serving architecture around PagedAttention and continuous batching and now combines prefix caching, chunked prefill, quantization, speculative decoding, and several forms of distributed inference [64, 131]. SGLang emphasizes reusable execution state through RadixAttention and has evolved into a high-performance runtime supporting structured outputs, cache-aware scheduling, prefill–decode disaggregation, speculative decoding, and large-scale expert parallelism [114, 152]. TensorRT-LLM targets performance-oriented NVIDIA deployments through a high-level PyTorch-native LLM API with paged KV-cache management, inflight batching, quantization, speculative decoding, parallelism, and disaggregated serving [88].

Table 19. Representative LLM training, post-training, orchestration, and inference frameworks. These systems occupy different layers and are often composed rather than substituted one-for-one.

Framework	Primary layer	Characteristic strengths	Typical role in an LLM stack
Megatron-LM / Megatron Core [86, 87, 119]	Large-scale training	Tensor, pipeline, data, expert, and context parallelism; GPU-optimized transformer/MoE building blocks	Low-level high-performance foundation-model training, especially when topology-aware multidimensional parallelism is central.
DeepSpeed [29, 109]	Distributed training	ZeRO family, 3D parallelism, sequence parallelism, MoE, optimizer/memory offload	Memory-efficient distributed training; frequently integrated underneath Transformers, Accelerate, or custom PyTorch loops.
Transformers + TRL [49, 139]	Model API / post-training	Broad model ecosystem plus SFT, DPO, GRPO, reward modeling, and other alignment trainers	High-level experimentation and post-training; can delegate distributed execution to DeepSpeed or other backends.
SWIFT [83]	Integrated fine-tuning stack	Pre-training/fine-tuning/alignment, PEFT, evaluation, quantization, inference, and deployment across many LLM/MLLM families	End-to-end practitioner workflow that reduces glue code between adaptation, evaluation, export, and serving backends.
Ray / Ray Train [5, 84]	Cluster orchestration	Worker/resource management, elastic job orchestration, framework-agnostic distributed execution	Schedules multi-node training and heterogeneous pipelines; commonly sits above PyTorch/Transformers/DeepSpeed rather than replacing their kernels.
slime [126]	RL post-training system	Connects Megatron training with SGLang rollout, custom data generation, reward/verifier feedback, and environment interaction	High-throughput reinforcement-learning loops in which rollout generation and policy updates must be coordinated continuously.
vLLM [64, 131]	Inference / serving	PagedAttention, continuous batching, prefix caching, chunked prefill, quantization, speculative decoding, distributed inference	General-purpose high-throughput serving with strong Hugging Face compatibility and an OpenAI-compatible interface.
SGLang [114, 152]	Inference / serving / rollout	RadixAttention prefix reuse, structured outputs, cache-aware scheduling, PD disaggregation, speculative decoding, expert parallelism	Serving long-prefix, structured, agentic, and RL-rollout workloads where state reuse and scheduling are first-class.
TensorRT-LLM [88]	NVIDIA-optimized inference	PyTorch-native LLM API, paged KV cache, inflight batching, quantization, speculative decoding, parallelism, and disaggregated serving	Performance-oriented deployment on NVIDIA GPUs, including multi-GPU/multi-node production serving and integration with NVIDIA inference infrastructure.

These engines increasingly overlap in feature lists, so their practical distinction is ecosystem and workload fit rather than one isolated optimization. vLLM is widely used as a general serving backend; SGLang is particularly natural for workloads with repeated prefixes, structured generation, agentic execution, or RL rollout; TensorRT-LLM is tightly optimized for NVIDIA deployment. Table 19 summarizes both the training and inference sides of the software landscape.

9.3 Training–Inference Co-Design: The Model Is No Longer a Standalone Artifact

The same technical decision now routinely moves cost between training and inference. MoE increases parameter capacity without activating all experts per token, but converts part of the compute problem into an expert-routing and all-to-all communication problem. GQA and MLA may leave pre-training quality nearly unchanged while sharply reducing KV-cache state during decoding. Long-context training creates serving workloads whose prefill can dominate latency and whose cached attention state can dominate memory. Quantization is usually an inference optimization, but quantization-aware training and low-precision pre-training increasingly blur that boundary. Multimodal models similarly create bursty token streams whose modality encoders, projectors, and generation heads stress different hardware resources.

This coupling changes how progress should be evaluated. A model architecture that achieves lower validation loss but requires substantially more active FLOPs, communication, or KV memory may be inferior at deployment scale. Conversely, a systems technique that improves throughput can make previously impractical reasoning budgets, context windows, or agent trajectories economically viable. The relevant object of study is therefore the end-to-end operating point over quality, training time, serving latency, memory footprint, energy, and cost.

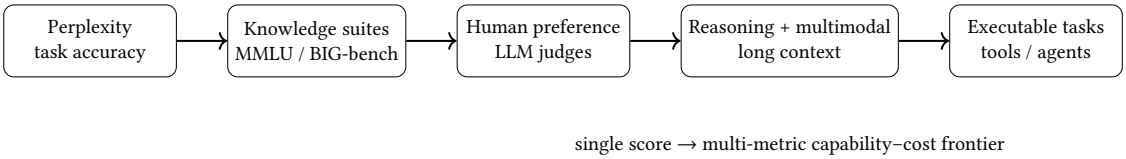


Fig. 19. Evaluation evolved from intrinsic language-model quality to increasingly interactive, executable, multimodal, and cost-aware assessment.

For information-access systems, this co-design is especially consequential. Retrieval-augmented generation, conversational search, and long-horizon agents alternate between model execution, retrieval, reranking, tool calls, and evidence accumulation. Their end-to-end latency depends on cache reuse and scheduling as much as on model FLOPs. Evaluation of LLM-centered information systems should therefore distinguish component quality from system quality: retrieval effectiveness, grounding, answer quality, time to first token, inter-token latency, and end-to-end task completion are different metrics and can trade off against one another.

Information-systems synthesis. Training and inference infrastructure determines whether an intelligent information system is economically deployable. Search assistants and recommenders may require high query throughput; chat requires low time-to-first-token and responsive decoding; long-context RAG requires large KV state; personalized assistants benefit from reusable prefixes and memory; and agents create bursty, asynchronous sequences of model calls and tool waits. Consequently, serving design affects not only cost but also information quality: latency limits how much evidence can be retrieved or verified, memory limits how much state can be preserved, and scheduling determines whether expensive reasoning can be allocated selectively. Model training and inference are therefore part of information-system design, not an implementation detail beneath it.

10 Evaluation: From Perplexity to Dynamic, Interactive Benchmarks

Evaluation has evolved almost as rapidly as LLM capability. Our benchmark selection follows the dedicated evaluation cluster in the curated Awesome-LLM-papers collection and also draws evaluation tasks from its code, mathematics, multimodal, and application categories, because modern LLM evaluation increasingly crosses these boundaries [41]. A benchmark that is discriminative for one generation can saturate in the next, and public test sets can leak into training corpora. Moreover, contemporary LLMs are not only text predictors: they reason, see images, use tools, write software, and act in environments. Evaluation therefore expanded from intrinsic language-model metrics to a portfolio of knowledge, reasoning, preference, multimodal, long-context, coding, tool-use, agent, safety, and efficiency tests. Table 20 and Figure 19 summarize this transition. Figure 20 gives a practical taxonomy of what must be specified in a modern evaluation, and Table 21 collects representative tasks, datasets, metrics, protocols, and caveats that recur in contemporary LLM papers.

10.1 From Reference-Based NLP Metrics to LLM Evaluation Protocols

The evaluation literature inherited several important ideas from pre-LLM NLP. BLEU compares machine translations with references through modified n-gram precision [94], while ROUGE emphasizes overlap-oriented recall for summarization [71]. Later semantic metrics such as BERTScore replace exact lexical matching with contextual-embedding similarity

Table 20. Evolution of LLM evaluation targets.

Evaluation class	Representative examples	What it measures	Main validity risk
Intrinsic / knowledge	Perplexity, MMLU	Language modeling and broad knowledge	Weak correlation with assistant behavior; contamination
Reasoning / code	MATH, HumanEval, GPQA	Multi-step reasoning and executable correctness	Targeted post-training and test-time budget differences
Human preference	MT-Bench, Arena-style voting	Helpfulness and conversational quality	Judge bias, verbosity bias, non-stationary users
Long context	LongBench, RULER	Retrieval/integration over long sequences	Synthetic needles may overstate real document reasoning
Multimodal	MMMU, OCR/chart/video suites	Cross-modal perception and reasoning	Preprocessing and language-prior shortcuts
Software/agents	SWE-bench, tool/function suites	Environment interaction and task completion	Scaffold dependence and environment drift
Safety/efficiency	Red teaming, latency/cost metrics	Deployment risk and resource efficiency	Protocol variation and incomplete threat models

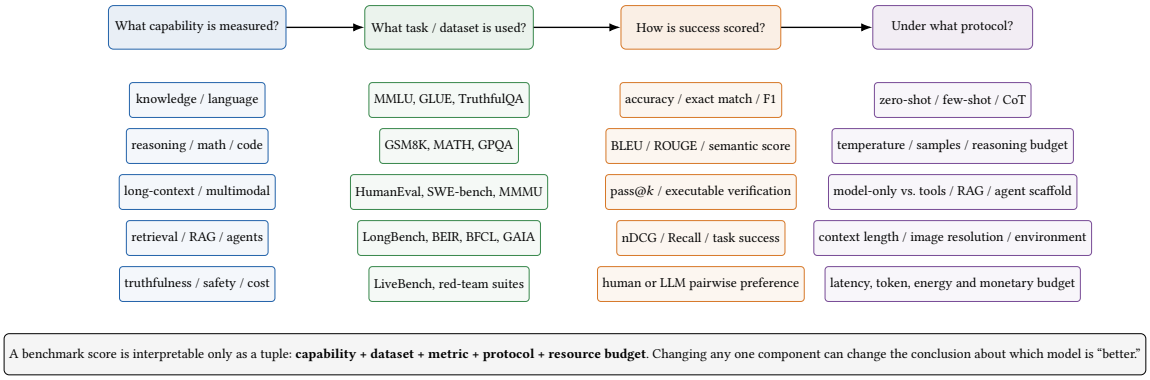


Fig. 20. A practical taxonomy for LLM evaluation. Modern evaluation is not a single benchmark leaderboard: a valid comparison must specify the capability, task/data, metric, inference protocol, tools or scaffolding, and resource budget.

[149]. These metrics remain useful for constrained generation tasks, but they are often poorly matched to open-ended assistants because many semantically correct responses can have little surface overlap with a reference.

Benchmark suites then shifted attention from one output string to a collection of capabilities. GLUE aggregated nine language-understanding tasks into a common evaluation platform [132]; MMLU later made broad academic and professional knowledge a standard LLM benchmark [42]. MMLU-Pro raises the difficulty and reduces prompt sensitivity by introducing more reasoning-intensive questions and a larger answer set [136]. The historical progression is therefore from *reference similarity*, to *task-specific correctness*, to *multi-capability evaluation under an explicit inference protocol.*

For contemporary LLMs, the protocol is part of the metric. A reported score should specify at minimum the prompt template, zero/few-shot setting, use of chain-of-thought, sampling temperature, number of samples, maximum output tokens, reasoning or test-time-compute budget, tool or retrieval access, and whether the number is model-only or produced by an agent scaffold. Without these controls, scores from the same nominal benchmark can be incomparable.

Table 21. Common LLM evaluation tasks, datasets/benchmarks, and metrics. The table combines evaluation works highlighted in the Awesome-LLM-papers collection [41] with benchmark families that have become standard in recent LLM papers. Metrics should be interpreted together with the prompting, sampling, tool, and test-time-compute protocol.

Capability / task	Representative dataset or benchmark	Common metric(s)	Typical evaluation protocol	Main caveat
Classical text generation	BLEU [94], ROUGE [71], BERTScore [149]	BLEU, ROUGE-n/L, semantic precision/recall/F1	Compare generated text with one or more references	Lexical overlap can penalize valid open-ended responses; learned metrics inherit encoder bias
General NLU	GLUE [132]	Accuracy, F1, Matthews correlation, aggregate score	Task-specific prediction over nine NLU datasets	Largely saturated for frontier LLMs; not designed for generative assistants
Broad knowledge	MMLU [42], MMLU-Pro [136]	Multiple-choice accuracy	Zero/few-shot or CoT; exact option extraction	Public questions can be contaminated; scores depend on prompt and reasoning budget
Math word problems	GSM8K [20]	Exact-match answer accuracy, pass@k	Direct answer or chain-of-thought; optionally sample multiple solutions	Short grade-school problems do not fully characterize advanced mathematical reasoning
Competition mathematics	MATH [43]	Exact match / equivalence, pass@k	Free-form derivation followed by answer normalization or symbolic checking	Strong sensitivity to answer parser and test-time sampling
Expert reasoning	GPQA [110]	Multiple-choice accuracy	Direct or CoT reasoning on graduate-level expert questions	Small, difficult set; repeated public use can lead to targeted optimization
Instruction following	IFEval [154]	Prompt-level / instruction-level strict and loose accuracy	Deterministically verify constraints such as format, length, and required phrases	Captures verifiable constraints, not the full semantics of helpfulness
Truthfulness / factuality	TruthfulQA [72]	Truthful, informative, MC accuracy; judge-based variants	Generate or select answers to misconception-inducing questions	Truthfulness judgments and model-based graders can be subjective or domain-sensitive
Open-ended chat quality	MT-Bench [151], Chatbot Arena [17]	LLM-judge score; pairwise win rate / Bradley–Terry-style rating	Multi-turn answers judged by strong LLMs or humans	Position, style, verbosity and evaluator-model biases; rankings drift over time
Code generation	HumanEval [15], MBPP [7]	pass@1, pass@k	Execute generated functions against hidden unit tests	Limited test coverage can accept incorrect programs; benchmark saturation
Repository-level software engineering	SWE-bench [54]	Resolved issue rate	Agent edits real repositories and runs tests	Strong scaffold, environment, retrieval, and tool dependence
Long-context understanding	LongBench [10], RULER [46]	Task-specific F1/EM/accuracy as a function of length	Evaluate retrieval, QA, summarization, aggregation at increasing context lengths	Maximum advertised context does not imply effective use; synthetic retrieval can be easier than integration
Multimodal reasoning	MMMU [145]	Accuracy	Questions combine images with expert-domain text reasoning	Image resolution, crop strategy, OCR and prompt format materially affect scores
Function calling / tool use	BFCL [95]	Function-selection and argument accuracy; AST/execution matching	Single, parallel, and multi-step function calls with schema constraints	Tool schemas and formatting conventions can dominate results
Interactive agents	AgentBench [75], GAIA [79]	Environment/task success, level-wise accuracy, action efficiency	Multi-turn interaction with OS, web, DB, tools, browsing, or multimodal resources	Environment drift, stochastic services, hidden scaffolding, and high evaluation cost
Information retrieval	BEIR [125]	nDCG@k, Recall@k, MAP, MRR	Retrieve/rank documents over heterogeneous zero-shot IR datasets	Retrieval score alone does not measure downstream answer faithfulness
RAG / grounded generation	RAGAS [32]; end-to-end RAG suites	Context precision/recall, faithfulness, answer relevance/correctness	Evaluate retriever and generator separately and jointly	LLM-based graders can introduce evaluator bias; reference-free scores need calibration
Dynamic / contamination-aware general evaluation	LiveBench [138]	Objective accuracy aggregated across categories	Frequently refreshed questions in reasoning, math, coding, language, data analysis, and instruction following	Longitudinal comparison changes as the benchmark is refreshed
Safety and robustness	Red-team suites, adversarial prompts, policy-specific tests	Attack success rate, refusal / compliance rate, toxicity or harm measures	Evaluate benign/unsafe pairs, jailbreaks, distribution shifts and adversarial inputs	Threat models and policies differ substantially across deployments
Efficiency / deployment	Throughput, time-to-first-token, inter-token latency, memory, energy, cost	tokens/s, latency, peak memory, joules/token, cost/query	Measure at specified hardware, batch size, context and output length	Results are hardware- and serving-stack-dependent; capability comparisons can be confounded by budget

10.2 Perplexity and Task Accuracy

Pre-training historically relied on held-out loss or perplexity because these metrics directly measure next-token prediction and are cheap to compute at scale. They remain useful for optimization and scaling-law analysis, but they are insufficient proxies for assistant utility: two models with similar validation loss may differ substantially in instruction following, reasoning, factuality, or safety after post-training.

Task benchmarks therefore became the dominant external evaluation. MMLU evaluates broad academic and professional knowledge across 57 subjects [42], while MMLU-Pro increases difficulty and emphasizes reasoning [136]. GSM8K targets grade-school mathematical word problems [20]; MATH moves to competition-style mathematics [43]; and GPQA targets graduate-level expert questions [110]. For instruction following, IFEval uses automatically verifiable constraints instead of subjective preference labels [154]. In code generation, HumanEval and MBPP use executable unit tests, making pass@k a natural metric [7, 15]. These benchmarks made progress legible, but their fixed public nature created saturation and contamination risks.

10.3 Holistic and Multi-Metric Evaluation

HELM argued that evaluation should cover multiple scenarios and metrics rather than a single leaderboard number [69]. Accuracy can trade off against calibration, robustness, bias, toxicity, and efficiency. This perspective is particularly important for industrial models because deployment choices depend on latency, cost, reliability, and safety as well as raw capability.

The multi-metric view also exposes hidden assumptions in benchmark aggregation. Averaging scores across tasks implicitly weights them equally; proprietary model reports often select different prompt formats, few-shot settings, or subsets. Reproducible survey comparison therefore requires recording protocol details rather than copying headline numbers.

10.4 Chat and Human-Preference Evaluation

Instruction-following models raised a new question: how should open-ended answers be scored? Human pairwise preference became common, but large-scale human evaluation is expensive and inconsistent. LLM-as-a-judge approaches such as MT-Bench and human-preference platforms such as Chatbot Arena provided scalable alternatives, using strong models or crowd pairwise votes to compare conversational quality [17, 151]. These methods better reflect style and helpfulness than exact-match benchmarks, but introduce judge bias, position bias, verbosity preferences, and sensitivity to the evaluator model.

Preference evaluation is also non-stationary. As users become accustomed to stronger systems, the meaning of "helpful" changes. Arena-style rankings therefore measure a moving product distribution rather than a fixed scientific task.

10.5 Harder Reasoning, Instruction-Following, and Truthfulness Benchmarks

Benchmark saturation motivated tests designed to resist shallow pattern matching. TruthfulQA explicitly measures whether models reproduce common human misconceptions rather than factual answers [72], while IFEval measures compliance with objectively checkable instructions [154]. These datasets illustrate an important expansion of evaluation: capability is not only whether a model *knows* an answer, but whether it follows the requested behavioral contract and avoids confidently reproducing false beliefs.

GPQA uses graduate-level questions written and validated by domain experts; newer math sets emphasize proof-like or competition reasoning; and contamination-aware benchmarks refresh questions over time. The main trend is toward tasks whose solutions require multi-step inference and whose answer space admits objective verification.

However, harder static benchmarks still face a validity problem: improved scores may reflect benchmark-specific post-training or test-time sampling rather than general reasoning. Reporting pass@k, majority vote, or tool-assisted scores alongside single-sample accuracy is therefore essential when models use large inference-time budgets.

10.6 Information Retrieval, Grounding, and Recommendation Evaluation

LLM evaluation for information access must separate the retrieval problem from the generation problem. Retrieval effectiveness is still measured with classical ranking metrics such as Recall@k, MRR, MAP, and nDCG, because a generator cannot reliably ground an answer in evidence it never receives. BEIR provides a heterogeneous zero-shot retrieval suite spanning multiple domains and retrieval settings [125]. Dense retrievers such as DPR and late-interaction systems such as ColBERT therefore remain relevant baselines even when the visible product is a generative assistant [57, 58]. End-to-end RAG evaluation adds answer correctness, faithfulness to evidence, citation precision/recall, robustness to irrelevant passages, and the latency or monetary cost of retrieval and reranking. RAGAS formalizes several reference-free dimensions such as context relevance and faithfulness, illustrating the move toward component-wise evaluation of retriever and generator behavior [32].

Adaptive retrieval complicates evaluation further because retrieval frequency becomes a learned policy. Self-RAG, for example, can retrieve on demand rather than at every generation step [6]. Two systems with similar answer accuracy

can therefore have very different retrieval cost, evidence coverage, and failure behavior. Evaluations should report both utility and resource use, including how often retrieval is invoked and whether citations actually support the generated claims.

Recommendation and ranking introduce another family of metrics. LLM-based rankers should be evaluated with established ranking metrics such as nDCG and MRR, but also for popularity bias, position bias, calibration, personalization, and sensitivity to candidate ordering. The zero-shot ranking results of Hou et al. illustrate both the promise and these biases [45]. For conversational recommendation and search, interaction-level outcomes—clarification quality, session success, diversity, and user effort—become as important as one-shot relevance.

10.7 Long-Context and Retrieval Evaluation

Long-context claims require dedicated evaluation because a nominal context window does not imply effective use. LongBench evaluates multiple long-document tasks, while RULER constructs controlled retrieval and reasoning tasks across sequence lengths [10, 46]. "Lost in the Middle" further showed strong position effects in long contexts [74]. Together these results motivate reporting quality as a function of context length rather than a single maximum-window number.

Long-context evaluation should also separate retrieval from integration. Needle-in-a-haystack tasks can test whether a model can recover a fact, but not whether it can synthesize contradictory evidence distributed across many documents. Practical long-context evaluation increasingly includes multi-hop, aggregation, and document-level reasoning.

10.8 Multimodal Evaluation

Multimodal models require tests for visual question answering, OCR, charts, documents, spatial grounding, video understanding, and audio. MMMU is representative of expert-level multimodal reasoning across disciplines [145]. The challenge is that modality-specific preprocessing can dominate results: image resolution, number of crops, frame sampling, OCR tools, and prompt formatting may materially change accuracy.

A second challenge is hallucination. A model may answer correctly from language priors without grounding in the image. Good multimodal evaluation therefore includes counterfactual images, fine-grained localization, or tasks where visual evidence is necessary.

10.9 Code and Software-Engineering Evaluation

HumanEval scores completion of isolated programming functions with unit tests [15]. SWE-bench moves evaluation to repository-level software engineering [54]: a model receives a real issue and codebase and must produce a patch that passes tests. This shift is important because it converts coding evaluation from generation to interaction with a complex artifact. Agent scaffolding, repository retrieval, test execution, and iterative repair become part of the measured system.

As a result, "model evaluation" and "agent evaluation" start to merge. A base model may perform modestly without tools but strongly inside an optimized scaffold. Papers should therefore distinguish model-only, model-plus-tools, and full-agent results.

10.10 Tool-Use and Agent Evaluation

Function-calling benchmarks such as BFCL score whether the model selects correct functions and arguments under single or multiple calls [95]. More advanced agent benchmarks place models in web, operating-system, database, software, or multimodal environments and score terminal success. AgentBench spans multiple interactive environments

[75], while GAIA asks real-world questions that require combinations of reasoning, browsing, multimodal understanding, and tool use [79]. These evaluations are harder to reproduce because environments change and external services are stochastic.

Agent evaluation also needs process metrics. Task success alone does not reveal whether the agent took dangerous, expensive, or excessively long actions. Useful measures include action count, tool errors, recovery rate, wall-clock latency, token cost, irreversible side effects, and success under perturbations.

10.11 Contamination, Benchmark Gaming, and Dynamic Evaluation

As benchmark questions become widely available, they may enter pre-training, synthetic-data pipelines, or post-training corpora. Exact contamination is hard to audit for closed industrial models. Even absent direct leakage, developers can optimize on public benchmarks through prompt engineering or targeted data collection. The benchmark then becomes part of the training loop and loses value as an independent test.

Dynamic benchmarks, private test sets, procedurally generated tasks, and continuously refreshed human evaluations can reduce this problem. LiveBench operationalizes this idea with frequently refreshed, objectively scored tasks across reasoning, mathematics, coding, language, data analysis, and instruction following [138]. Yet dynamic benchmarks make longitudinal comparison harder because the test distribution itself evolves. The field therefore faces a tension between reproducibility and resistance to gaming.

10.12 Evaluation Must Include Cost

Reasoning models and agents make inference budgets highly variable. Comparing only accuracy can reward systems that use many more samples, longer chains of thought, or expensive tools. A more informative frontier plots task success against tokens, latency, energy, or monetary cost. For deployment, the relevant question is often not "Which model scores highest?" but "Which system reaches the required reliability at the lowest total cost?"

This motivates the later end-to-end utility perspective. Evaluation increasingly measures a system operating under heterogeneous resource constraints, not an isolated neural network under a fixed prompt.

10.13 A Minimum Reporting Protocol for LLM-Based Information Systems

A recurring source of irreproducibility is that two systems can report the same benchmark while using different retrieval access, reasoning budgets, tools, prompts, or serving policies. For LLM-centered information systems, evaluation should therefore report the operating conditions that generate the score. Table 22 proposes a minimum protocol covering the model and prompt, retrieval or recommendation pipeline, inference budget, tools and environment, and end-to-end outcomes. The protocol is intentionally lightweight: it does not prescribe one benchmark suite, but it makes clear which resources were available and where performance came from.

10.14 From Model Evaluation to Information-System Evaluation

For LLM-centered information systems, model-only accuracy is insufficient. A RAG system can fail because retrieval missed the evidence even when the generator is strong; a recommender can produce persuasive explanations while ranking poor items; a chatbot can answer correctly but violate latency or safety constraints; and an agent can reach the right final state through excessive, risky, or unauthorized actions. Evaluation should therefore decompose the pipeline into retrieval/ranking quality, evidence use and grounding, generation quality, interaction success, personalization, tool correctness, recovery, provenance, safety, latency, and cost, while also measuring end-to-end user or task outcomes.

Table 22. Recommended minimum reporting protocol for evaluating LLM-based intelligent information systems. The goal is to make results comparable across model-only, retrieval-augmented, tool-using, and agentic settings.

Dimension	Minimum information to report	Why it matters
Model and prompting	Model/version, decoding settings, prompt template, system prompt, context limit, and any fine-tuning	Prevents hidden prompt or version differences from being mistaken for model capability.
Retrieval / recommendation	Corpus/index version, retriever/ranker, k, freshness, personalization inputs, and retrieval metrics	Separates failures of information access from failures of generation or reasoning.
Inference budget	Reasoning mode, token/sample budget, self-consistency/search, cache reuse, and stopping policy	Makes quality comparisons meaningful when test-time compute differs.
Tools and environment	Available tools, permissions, environment version, external services, and failure/retry policy	Distinguishes model-only results from scaffolded or agentic system performance.
End-to-end outcomes	Task success or user utility together with latency, cost, grounding/provenance, safety, and recovery metrics	Captures the operating point experienced by users rather than a single component score.

This shift is especially important for information-systems research because the object of evaluation is increasingly the complete information service rather than the isolated foundation model.

11 New Perspectives: Trajectory Scaling, Orchestration, and End-to-End Utility

The historical record suggests that "scaling" is no longer well described by parameter count or pre-training FLOPs alone. Modern intelligent information systems combine a foundation model with retrieval, ranking, memory, tools, verifiers, multimodal interfaces, and serving infrastructure. We therefore introduce three analytical perspectives that follow from the technical evolution surveyed above: the *successful trajectory as a scaling unit*, *orchestration scaling across model and external information resources*, and *end-to-end information-system utility*. These are intended as testable organizing hypotheses rather than fitted scaling laws.

11.1 Perspective 1: The Successful Trajectory as a Scaling Unit

For pre-training, the natural unit is the token; for supervised tuning, the demonstration; for preference learning, the comparison. For reasoning and agents, a more meaningful unit is increasingly the *trajectory*: a sequence of internal computations, retrievals, tool actions, observations, and revisions that terminates in success or failure.

This perspective changes both data and systems design. A failed trajectory can contain useful local decisions, while a successful trajectory may contain wasteful loops or unsafe actions. Credit assignment should therefore operate within trajectories rather than only at terminal reward. Trajectory generation is also asynchronous and variable-length, which changes batching, scheduling, replay, and evaluation. For AI search and retrieval agents, the trajectory may include query reformulation, document selection, evidence synthesis, verification, and user-facing response; for coding agents it may include repository search, edits, execution, diagnosis, and repair.

11.2 Toward Trajectory Scaling Laws

A future scaling theory may relate task success to the number, diversity, cost, and quality of trajectories used in post-training. Candidate variables include verifier accuracy, horizon length, branching factor, environment stochasticity, recovery opportunities, and the fraction of trajectories containing informative feedback. Unlike text tokens, trajectories are expensive, correlated, and policy-dependent, so simple scaling by count is unlikely to be sufficient.

The empirical question is therefore not merely whether "more trajectories help," but which trajectory properties predict generalization to new environments. One testable hypothesis is that coverage of failure-and-recovery patterns will predict long-horizon reliability better than raw trajectory count. Establishing or rejecting such relationships would connect foundation-model scaling with reinforcement learning, interactive information retrieval, and sequential decision making.

11.3 Perspective 2: From Model Scaling to Orchestration Scaling

A second transition is from scaling a single model to scaling the quality of orchestration among heterogeneous components. Search systems decide when to retrieve and rerank; RAG systems decide which evidence to place in context; personalized assistants decide what to store or recall from memory; reasoning systems decide when to verify; and agents decide when to invoke tools or request additional information. These choices determine what information reaches the model and which actions follow from its output.

This suggests that an increasingly important capability frontier is *orchestration quality*: how effectively a system routes each request among parametric knowledge, external retrieval, persistent memory, specialized tools, verifiers, and human interaction. A larger foundation model can reduce the need for some external operations, but better retrieval or verification can also allow a smaller model to outperform a larger one on grounded tasks. The research problem is therefore not simply component selection but learning policies that choose the right component, at the right stage, with the right evidence and stopping condition.

This perspective is especially relevant to information systems because retrieval, ranking, recommendation, provenance, access control, and freshness cannot be reduced to language modeling. They are external system functions that must remain inspectable and controllable. A testable hypothesis is that improvements in learned routing and evidence selection will produce larger gains on dynamic information-access tasks than equivalent increases in base-model size once the model reaches a sufficient competence threshold.

11.4 Perspective 3: Intelligent Information Systems as End-to-End Utility Optimization

The trajectory and orchestration views become most useful when the optimization target is the complete information service. Search and recommendation select information; RAG integrates evidence; reasoning assistants transform information into decisions; agents connect decisions to external actions; and serving systems determine whether the full loop remains responsive and economical. The user experiences the combined result rather than any single component score.

We therefore argue that the relevant objective is an end-to-end utility frontier over information quality, task success, latency, cost, provenance, safety, and risk. A larger model can be inferior to a smaller model paired with better retrieval, fresher data, stronger verification, or a better interaction policy. Conversely, a strong retriever can be undermined by a generator that ignores evidence, and a high-accuracy agent can be unacceptable if it reaches the answer through excessive or unauthorized actions.

This view reconnects modern foundation-model research with enduring information-systems questions. It motivates experiments that vary model, retrieval, memory, verification, and tool policies jointly and report the resulting system-level frontier rather than a single model score. It also shifts the unit of engineering from the checkpoint to the complete information-processing loop: acquire information, select evidence, reason over it, act when necessary, verify outcomes, and preserve useful state for the next interaction.

12 Open Challenges and Future Directions

The evolutionary view in this survey suggests that the remaining problems are increasingly cross-layer. They cannot be solved by scaling a base model in isolation because they involve information quality, system state, resource allocation, user interaction, and governance. We organize the agenda around seven coupled challenges.

12.1 Evidence, Transparency, and Data Governance

Industrial models define much of the frontier but often disclose incomplete information about training data, compute, post-training, and evaluation. This makes causal attribution difficult: a benchmark gain may come from architecture, data, targeted post-training, retrieval, tools, or a larger test-time budget. Future model reports should distinguish total and active parameters, describe major training-data domains and context-training phases, document post-training stages, and report inference budgets and tool access for benchmarks.

Data governance is inseparable from this transparency problem. High-quality public text is finite, while synthetic data can be generated without obvious limit. Scaling synthetic data without narrowing diversity or amplifying model errors requires verification, source mixing, uncertainty filtering, and human-authored anchor data. Provenance is also necessary for copyright, privacy, contamination analysis, and reproducibility. For information systems, these concerns extend beyond the training corpus to retrieved evidence, user histories, recommendation logs, and persistent memory.

12.2 Adaptive Reasoning, Efficiency, and Sustainable Serving

Test-time reasoning can improve difficult tasks while making latency and cost unpredictable. Future systems should learn calibrated stopping policies, recognize when retrieval or tools are more efficient than internal deliberation, and allocate multiple samples only when uncertainty justifies them. The target should be a Pareto frontier over correctness, latency, monetary cost, and energy rather than maximum benchmark score.

This problem is linked to serving design. Sparse models, quantization, cache compression, speculative decoding, prefix reuse, and specialized accelerators can reduce per-query cost, but increased demand may offset those gains. Reporting hardware-normalized throughput, energy, and memory alongside quality would make system improvements easier to compare. A particularly important research direction is adaptive routing across models and reasoning modes: easy information requests should not pay the same computational price as hard analytical tasks.

12.3 Memory, Personalization, and Continual Information Systems

Long context is not persistent memory. Intelligent information systems need to retain useful facts across sessions, forget obsolete information, respect user preferences, and distinguish trusted from untrusted sources. External memory is editable and auditable but can be noisy; parametric memory is compact but difficult to update; user profiles are useful for personalization but create privacy and governance obligations.

The open problem is therefore not only memory capacity but *memory policy*: what should be stored, where it should live, when it should be refreshed, and who is allowed to access it. Search, recommendation, and conversational systems will increasingly need hybrid memory architectures that combine retrieval indexes, structured profiles, summaries, episodic traces, and model context while preserving provenance and deletion guarantees.

12.4 Reliable and Secure Agentic Information Systems

Agent errors compound with horizon length. A model that selects the correct action with high probability can still have low end-to-end success over dozens of steps. Reliability therefore requires verification, recovery, checkpointing, hierarchical planning, and safe rollback. Benchmarks should perturb environments and measure recovery rather than reward memorized scripts.

Security is equally important because tool-enabled models can alter external state. Prompt injection can arrive through retrieved documents, webpages, database fields, or tool output. Authentication and authorization cannot be delegated entirely to the model. Future agent stacks need least-privilege tool scopes, provenance-aware context, sandboxing, transaction-like safeguards, human confirmation for consequential actions, and auditable traces. In information-system terms, agents transform access control from a peripheral concern into part of the reasoning loop itself.

12.5 Grounded Multimodal Information Access

Multimodal systems broaden information access but also broaden the space of possible grounding failures. A model may produce a fluent answer using language priors while ignoring the relevant region of an image, time segment of a video, or state of a user interface. Better training and evaluation should therefore require explicit evidence attribution across modalities.

This creates new systems problems. Documents combine text, layout, figures, and tables; video and audio are temporal; interfaces are dynamic. Future work needs cross-modal indexing, synchronized retrieval, compressed temporal representations, and provenance mechanisms that identify which visual, acoustic, or textual evidence supported a decision. The challenge is not merely multimodal generation but trustworthy access to heterogeneous information.

12.6 Evaluation, Benchmark Governance, and Causal Diagnosis

Static public benchmarks increasingly become training targets. Dynamic or private evaluation reduces leakage but weakens reproducibility and longitudinal comparison. Agent benchmarks add environment drift, and RAG or recommendation evaluations depend on corpus, index, freshness, and user-state versions. Benchmark governance must therefore balance refresh, transparency, and versioning.

More importantly, end-to-end evaluation should support *causal diagnosis*. A poor answer may originate in retrieval, ranking, context construction, generation, tool choice, or serving constraints. Reporting only final accuracy hides the failure source. Future evaluation suites should combine component metrics with end-to-end utility and record the resource protocol under which the result was produced, following the reporting principles in Section 10.

12.7 Toward Predictive Theories of End-to-End Information-System Scaling

Classical scaling laws are powerful because they reduce complex behavior to a few variables. Modern LLM-centered systems add sparsity, heterogeneous data, long context, retrieval, test-time search, persistent memory, and external tools. A useful theory would predict not only how loss changes with compute, but how end-task utility changes when resources are reallocated among these components.

This creates a research program at the intersection of foundation models and information systems. For a given workload, when should a system rely on a larger base model, fresher data, better retrieval, more context, additional reasoning, stronger verification, or an external tool? How should those choices change with latency constraints, user

risk, personalization, or information freshness? Answering these questions would turn qualitative co-design principles into a predictive theory of intelligent information-system architecture and orchestration.

13 Conclusion

This survey examined the evolution of large language models through the lens of intelligent information systems. The central argument is not that an LLM replaces an information system, but that it is becoming a powerful semantic reasoning and interaction layer within one. Retrieval indexes, rankers, recommender models, databases, user profiles, memory stores, permissions, execution environments, and monitoring remain essential. What has changed is the model's ability to connect these components through natural-language understanding, evidence synthesis, multimodal interpretation, planning, and tool use.

The historical analysis answers the first research question by showing that progress has come from a sequence of bottleneck shifts rather than one uninterrupted scaling curve. Dense parameter growth created training and serving constraints; compute-optimal training elevated data allocation; post-training separated behavior shaping from representation learning; long context and memory expanded information bandwidth while exposing attention and KV-state costs; MoE separated stored capacity from active compute; multimodality widened the information interface; and reasoning and agents moved substantial computation from pre-training into inference and environment interaction. Each transition improved capability while relocating cost or complexity to another layer.

The second research question concerns information-system consequences. Architecture determines how economically a system can preserve and process evidence. Pre-training shapes semantic coverage and domain competence. Post-training determines how evidence is followed, synthesized, explained, and converted into decisions. Retrieval and recommendation connect the model to external collections and user needs. Long context and memory determine how information is selected and retained over time. Multimodality expands the space of searchable and actionable information objects. Agents add action, feedback, and recovery. Training and inference systems determine whether these capabilities remain practical at real-world latency, throughput, and cost.

The third research question is therefore fundamentally about co-design. Modern capability cannot be attributed to base-model scale alone. A smaller model with better retrieval, fresher information, stronger verification, or a more appropriate reasoning policy can dominate a larger model on a specific workload. Conversely, larger context or more reasoning can improve quality while violating latency or cost constraints. The perspectives developed in this paper make this shift explicit: interactive systems increasingly scale through successful trajectories and through better orchestration among models, retrieval, memory, verifiers, tools, and environments. The relevant object is therefore a sequence of information-seeking, reasoning, action, verification, and recovery rather than an isolated response.

The fourth research question concerns evaluation. Once retrieval, recommendation, tools, memory, and variable inference budgets enter the pipeline, model-only accuracy becomes an incomplete measure. Evaluation must identify where information was obtained, what evidence was selected, how much computation was used, which tools were available, and whether the complete system produced a useful, grounded, safe, timely, and cost-effective outcome. This is why we advocate reporting both component metrics and end-to-end information-system utility.

Taken together, these findings suggest a shift in the optimization target for LLM research. The question is no longer only how to build a more capable language model. It is how to design a better information service in which models, retrieval and recommendation, memory, multimodal interfaces, tools, infrastructure, and governance allocate resources coherently around user and task needs. For intelligent information systems, the most consequential advances may

therefore come not from any single model component, but from learning how to route information and computation across the full stack while preserving provenance, efficiency, personalization, security, and trust.

References

[1] 01.AI, Alex Young, Bei Chen, Chao Li, et al. 2024. Yi: Open Foundation Models by 01.AI. *arXiv preprint arXiv:2403.04652* (2024).

[2] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. 2023. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. *Proceedings of EMNLP* (2023).

[3] Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katie Millican, Malcolm Reynolds, et al. 2022. Flamingo: A Visual Language Model for Few-Shot Learning. *Advances in Neural Information Processing Systems* (2022).

[4] Ebtesam Almazrouei, Hamza Alobeidli, Abdulaziz Alshamsi, et al. 2023. The Falcon Series of Open Language Models. *arXiv preprint arXiv:2311.16867* (2023).

[5] Anyscale. 2026. Ray Train: Distributed Training at Scale. Software documentation. https://docs.ray.io/en/latest/train/overview.html

[6] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2024. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. In *The Twelfth International Conference on Learning Representations*.

[7] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc V. Le, and Charles Sutton. 2021. Program Synthesis with Large Language Models. *arXiv preprint arXiv:2108.07732* (2021).

[8] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, et al. 2023. Qwen Technical Report. *arXiv preprint arXiv:2309.16609* (2023).

[9] Jinze Bai, Shuai Bai, Shusheng Yang, Shijie Wang, Sinan Tan, Peng Wang, Junyang Lin, Chang Zhou, and Jingren Zhou. 2023. Qwen-VL: A Versatile Vision-Language Model for Understanding, Localization, Text Reading, and Beyond. *arXiv preprint arXiv:2308.12966* (2023).

[10] Yushi Bai et al. 2024. LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*. 3119–3137.

[11] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. 2022. Constitutional AI: Harmlessness from AI Feedback. *arXiv preprint arXiv:2212.08073* (2022).

[12] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems*.

[13] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. 2024. Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads. *Proceedings of ICML* (2024).

[14] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. 2023. Accelerating Large Language Model Decoding with Speculative Sampling. *arXiv preprint arXiv:2302.01318* (2023).

[15] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* (2021).

[16] Shouyuan Chen, Sherman Wong, Liangjian Chen, and Yuandong Tian. 2023. Extending Context Window of Large Language Models via Positional Interpolation. *arXiv preprint arXiv:2306.15595* (2023).

[17] Wei-Lin Chiang et al. 2024. Chatbot Arena: An Open Platform for Evaluating LLMs by Human Preference. In *International Conference on Machine Learning*.

[18] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. 2022. PaLM: Scaling Language Modeling with Pathways. *arXiv preprint arXiv:2204.02311* (2022).

[19] Paul F. Christiano, Jan Leike, Tom B. Brown, Miljan Martic, Shane Legg, and Dario Amodei. 2017. Deep Reinforcement Learning from Human Preferences. In *Advances in Neural Information Processing Systems*.

[20] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training Verifiers to Solve Math Word Problems. *arXiv preprint arXiv:2110.14168* (2021).

[21] Tri Dao. 2023. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. *arXiv preprint arXiv:2307.08691* (2023).

[22] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *Advances in Neural Information Processing Systems*.

[23] DeepSeek-AI. 2024. Auxiliary-Loss-Free Load Balancing Strategy for Mixture-of-Experts. *arXiv preprint* (2024).

[24] DeepSeek-AI. 2024. DeepSeek LLM: Scaling Open-Source Language Models with Longtermism. *arXiv preprint* (2024).

[25] DeepSeek-AI. 2024. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. *arXiv preprint arXiv:2405.04434* (2024).

[26] DeepSeek-AI. 2024. DeepSeek-V3 Technical Report. *arXiv preprint arXiv:2412.19437* (2024).

[27] DeepSeek-AI. 2024. DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models. *arXiv preprint arXiv:2401.06066* (2024).

[28] DeepSeek-AI. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *arXiv preprint arXiv:2501.12948* (2025).

[29] DeepSpeed Team. 2026. DeepSpeed: Extreme Speed and Scale for Deep Learning Training. GitHub repository. https://github.com/deepspeedai/DeepSpeed

[30] Tim Dettmers et al. 2023. QLoRA: Efficient Finetuning of Quantized LLMs. In *Advances in Neural Information Processing Systems*.

[31] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. 2024. The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783* (2024).

[32] Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. 2023. RAGAS: Automated Evaluation of Retrieval Augmented Generation. *arXiv preprint arXiv:2309.15217* (2023).

[33] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. *Journal of Machine Learning Research* 23 (2022).

[34] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. 2023. Retrieval-Augmented Generation for Large Language Models: A Survey. *arXiv preprint arXiv:2312.10997* (2023).

[35] Gemini Team. 2023. Gemini: A Family of Highly Capable Multimodal Models. *arXiv preprint arXiv:2312.11805* (2023).

[36] Gemini Team. 2024. Gemini 1.5: Unlocking Multimodal Understanding across Millions of Tokens of Context. *arXiv preprint arXiv:2403.05530* (2024).

[37] Gemini Team. 2025. Gemini 2.5: Pushing the Frontier with Advanced Reasoning, Multimodality, Long Context, and Next Generation Agentic Capabilities. *arXiv preprint* (2025).

[38] Gemma Team. 2024. Gemma 2: Improving Open Language Models at a Practical Size. *arXiv preprint arXiv:2408.00118* (2024).

[39] GLM-4.5 Team. 2025. GLM-4.5: Agentic, Reasoning, and Coding (ARC) Foundation Models. *arXiv preprint arXiv:2508.06471* (2025).

[40] GLM-5 Team. 2026. GLM-5: from Vibe Coding to Agentic Engineering. *arXiv preprint arXiv:2602.15763* (2026).

[41] Yulong Gu. 2026. Awesome-LLM-papers: Awesome Papers in Large Language Models. GitHub repository. https://github.com/guyulongcs/Awesome-LLM-papers Accessed September 2026.

[42] Dan Hendrycks et al. 2021. Measuring Massive Multitask Language Understanding. In *International Conference on Learning Representations*.

[43] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring Mathematical Problem Solving With the MATH Dataset. In *Advances in Neural Information Processing Systems*.

[44] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. 2022. Training Compute-Optimal Large Language Models. *arXiv preprint arXiv:2203.15556* (2022).

[45] Yupeng Hou, Junjie Zhang, Zihan Lin, Hongyu Lu, Ruobing Xie, Julian J. McAuley, and Wayne Xin Zhao. 2024. Large Language Models are Zero-Shot Rankers for Recommender Systems. In *Advances in Information Retrieval: 46th European Conference on Information Retrieval*. 364–381. doi:10.1007/978-3-031-56060-6_24

[46] Cheng-Ping Hsieh et al. 2024. RULER: What's the Real Context Size of Your Long-Context Language Models? *arXiv preprint arXiv:2404.06654* (2024).

[47] Edward J. Hu et al. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations*.

[48] Yanping Huang, Yonglong Cheng, Ankur Bapna, Orhan Firat, Mia Chen, Dehao Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V. Le, Yonghui Wu, and zhifeng Chen. 2019. GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism. In *Advances in Neural Information Processing Systems*.

[49] Hugging Face. 2026. TRL: Transformers Reinforcement Learning. Software documentation. https://huggingface.co/docs/trl/

[50] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. 2024. Qwen2.5-Coder Technical Report. *arXiv preprint* (2024).

[51] Gautier Izacard, Patrick Lewis, Maria Lomeli, Lucas Hosseini, Fabio Petroni, Timo Schick, Jane Dwivedi-Yu, Armand Joulin, Sebastian Riedel, and Edouard Grave. 2023. Atlas: Few-shot Learning with Retrieval Augmented Language Models. *Journal of Machine Learning Research* 24, 251 (2023), 1–43.

[52] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, et al. 2023. Mistral 7B. *arXiv preprint arXiv:2310.06825* (2023).

[53] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, et al. 2024. Mixtral of Experts. *arXiv preprint arXiv:2401.04088* (2024).

[54] Carlos E. Jimenez et al. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *International Conference on Learning Representations*.

[55] Keller Jordan et al. 2025. Muon is Scalable for LLM Training. *arXiv preprint* (2025).

[56] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling Laws for Neural Language Models. *arXiv preprint arXiv:2001.08361* (2020).

[57] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense Passage Retrieval for Open-Domain Question Answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 6769–6781. doi:10.18653/v1/2020.emnlp-main.550

[58] Omar Khattab and Matei Zaharia. 2020. ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 39–48. doi:10.1145/3397271.3401075

[59] Kimi Team. 2025. Kimi k1.5: Scaling Reinforcement Learning with LLMs. *arXiv preprint* (2025).

[60] Kimi Team. 2025. Kimi K2: Open Agentic Intelligence. *arXiv preprint arXiv:2507.20534* (2025).

[61] Kimi Team. 2025. Kimi Linear: An Expressive, Efficient Attention Architecture. *arXiv preprint* (2025).

[62] Diederik P. Kingma and Jimmy Ba. 2015. Adam: A Method for Stochastic Optimization. In *International Conference on Learning Representations*.

[63] Taku Kudo and John Richardson. 2018. SentencePiece: A Simple and Language Independent Subword Tokenizer and Detokenizer for Neural Text Processing. In *Proceedings of EMNLP: System Demonstrations*.

[64] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of SOSP*.

[65] Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast Inference from Transformers via Speculative Decoding. *Proceedings of ICML* (2023).

[66] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*.

[67] Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. 2023. BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models. In *Proceedings of the 40th International Conference on Machine Learning*.

[68] Xiaoxi Li, Jiajie Jin, Yujia Zhou, Yuyao Zhang, Peitian Zhang, Yutao Zhu, and Zhicheng Dou. 2025. From Matching to Generation: A Survey on Generative Information Retrieval. *ACM Transactions on Information Systems* 43, 3, Article 83 (2025), 62 pages. doi:10.1145/3722552

[69] Percy Liang et al. 2023. Holistic Evaluation of Language Models. *Transactions on Machine Learning Research* (2023).

[70] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2023. Let's Verify Step by Step. *arXiv preprint arXiv:2305.20050* (2023).

[71] Chin-Yew Lin. 2004. ROUGE: A Package for Automatic Evaluation of Summaries. In *Text Summarization Branches Out*. 74–81.

[72] Stephanie Lin, Jacob Hilton, and Owain Evans. 2022. TruthfulQA: Measuring How Models Mimic Human Falsehoods. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*. 3214–3252. doi:10.18653/v1/2022.acl-long.229

[73] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. 2023. Visual Instruction Tuning. *Advances in Neural Information Processing Systems* (2023).

[74] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics* (2024).

[75] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. 2024. AgentBench: Evaluating LLMs as Agents. In *International Conference on Learning Representations*.

[76] Ilya Loshchilov and Frank Hutter. 2019. Decoupled Weight Decay Regularization. In *International Conference on Learning Representations*.

[77] Z. Lu et al. 2025. MoBA: Mixture of Block Attention for Long-Context LLMs. *Advances in Neural Information Processing Systems* (2025).

[78] Meta AI. 2025. The Llama 4 Herd: The Beginning of a New Era of Natively Multimodal AI Innovation. https://ai.meta.com/blog/llama-4-multimodal-intelligence/

[79] Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. 2023. GAIA: A Benchmark for General AI Assistants. *arXiv preprint arXiv:2311.12983* (2023).

[80] MiniMax. 2025. MiniMax-01: Scaling Foundation Models with Lightning Attention. *arXiv preprint arXiv:2501.08313* (2025).

[81] MiniMax. 2025. MiniMax-M1: Scaling Test-Time Compute Efficiently with Lightning Attention. *arXiv preprint arXiv:2506.13585* (2025).

[82] MiniMax. 2026. The MiniMax-M2 Series: Mini Activations Unleashing Max Real-World Intelligence. *arXiv preprint arXiv:2605.26494* (2026).

[83] ModelScope Community. 2026. SWIFT: Scalable lightWeight Infrastructure for Fine-Tuning. GitHub repository. https://github.com/modelscope/ms-swift

[84] Philipp Moritz, Robert Nishihara, Stephanie Wang, et al. 2018. Ray: A Distributed Framework for Emerging AI Applications. In *13th USENIX Symposium on Operating Systems Design and Implementation*.

[85] Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, et al. 2021. WebGPT: Browser-Assisted Question-Answering with Human Feedback. *arXiv preprint arXiv:2112.09332* (2021).

[86] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, Amar Phanishayee, and Matei Zaharia. 2021. Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM. *Proceedings of SC* (2021).

[87] NVIDIA. 2026. Megatron-LM and Megatron Core. GitHub repository. https://github.com/NVIDIA/Megatron-LM

[88] NVIDIA. 2026. TensorRT-LLM Documentation. Software documentation. https://nvidia.github.io/TensorRT-LLM/

[89] OpenAI. 2023. GPT-4 Technical Report. *arXiv preprint arXiv:2303.08774* (2023).

[90] OpenAI. 2024. GPT-4o System Card. OpenAI system card. https://openai.com/index/gpt-4o-system-card/

[91] OpenAI. 2024. OpenAI o1 System Card. OpenAI system card. https://openai.com/index/openai-o1-system-card/

[92] OpenAI. 2025. GPT-5 System Card. OpenAI system card. https://openai.com/index/gpt-5-system-card/

[93] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training Language Models to Follow Instructions with Human Feedback. In *Advances in Neural Information Processing Systems*.

[94] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. BLEU: A Method for Automatic Evaluation of Machine Translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*. 311–318. doi:10.3115/1073083.1073135

[95] Shishir G. Patil et al. 2025. Berkeley Function Calling Leaderboard. *arXiv preprint* (2025).

[96] Guilherme Penedo et al. 2024. The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale. *arXiv preprint arXiv:2406.17557* (2024).

[97] Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. 2023. YaRN: Efficient Context Window Extension of Large Language Models. *arXiv preprint arXiv:2309.00071* (2023).

[98] Boci Peng, Yun Zhu, Yongchao Liu, Xiaohe Bo, Haizhou Shi, Chuntao Hong, Yan Zhang, and Siliang Tang. 2026. Graph Retrieval-Augmented Generation: A Survey. *ACM Transactions on Information Systems* 44, 2, Article 35 (2026), 52 pages. doi:10.1145/3777378

[99] Ruoyu Qin et al. 2024. Mooncake: A KVCache-Centric Disaggregated Architecture for LLM Serving. *arXiv preprint* (2024).

[100] Zhen Qin et al. 2024. Various Lengths, Constant Speed: Efficient Language Modeling with Lightning Attention. *Proceedings of ICML* (2024).

[101] Qwen Team. 2024. Qwen2.5-Math Technical Report: Toward Mathematical Expert Model via Self-Improvement. *arXiv preprint* (2024).

[102] Qwen Team. 2024. Qwen2.5 Technical Report. *arXiv preprint arXiv:2412.15115* (2024).

[103] Qwen Team. 2025. Qwen2.5-Omni Technical Report. *arXiv preprint* (2025).

[104] Qwen Team. 2025. Qwen2.5-VL Technical Report. *arXiv preprint* (2025).

[105] Qwen Team. 2026. Qwen3.5-Omni Technical Report. *arXiv preprint arXiv:2604.15804* (2026).

[106] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. 2018. Improving Language Understanding by Generative Pre-Training. *OpenAI Technical Report* (2018).

[107] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language Models are Unsupervised Multitask Learners. *OpenAI Technical Report* (2019).

[108] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon, and Chelsea Finn. 2023. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. In *Advances in Neural Information Processing Systems*.

[109] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. ZeRO: Memory Optimizations Toward Training Trillion Parameter Models. (2020).

[110] David Rein et al. 2023. GPQA: A Graduate-Level Google-Proof Q&A Benchmark. *arXiv preprint arXiv:2311.12022* (2023).

[111] Teven Le Scao, Angela Fan, Christopher Akiki, et al. 2022. BLOOM: A 176B-Parameter Open-Access Multilingual Language Model. *arXiv preprint arXiv:2211.05100* (2022).

[112] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems*.

[113] Rico Sennrich, Barry Haddow, and Alexandra Birch. 2016. Neural Machine Translation of Rare Words with Subword Units. In *Proceedings of ACL*.

[114] SGLang Team. 2026. SGLang: High-Performance Serving Framework for Large Language and Multimodal Models. GitHub repository. https://github.com/sgl-project/sglang

[115] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, et al. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv preprint arXiv:2402.03300* (2024).

[116] Noam Shazeer. 2019. Fast Transformer Decoding: One Write-Head is All You Need. *arXiv preprint arXiv:1911.02150* (2019).

[117] Noam Shazeer. 2020. GLU Variants Improve Transformer. *arXiv preprint arXiv:2002.05202* (2020).

[118] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2017. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. *arXiv preprint arXiv:1701.06538* (2017).

[119] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. *arXiv preprint arXiv:1909.08053* (2019).

[120] Shaden Smith, Mostofa Patwary, Brandon Norick, et al. 2022. Using DeepSpeed and Megatron to Train Megatron-Turing NLG 530B, A Large-Scale Generative Language Model. *arXiv preprint arXiv:2201.11990* (2022).

[121] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. 2024. Scaling LLM Test-Time Compute Optimally Can Be More Effective than Scaling Model Parameters. *arXiv preprint* (2024).

[122] Nisan Stiennon, Long Ouyang, Jeffrey Wu, Daniel Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul F. Christiano. 2020. Learning to Summarize with Human Feedback. In *Advances in Neural Information Processing Systems*.

[123] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. 2024. RoFormer: Enhanced Transformer with Rotary Position Embedding. *Neurocomputing* (2024).

[124] Yi Tay, Vinh Q. Tran, Mostafa Dehghani, Jianmo Ni, Dara Bahri, Harsh Mehta, Zhen Qin, Kai Hui, Zhe Zhao, Jai Gupta, Tal Schuster, William W. Cohen, and Donald Metzler. 2022. Transformer Memory as a Differentiable Search Index. In *Advances in Neural Information Processing Systems*, Vol. 35.

[125] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. 2021. BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models. In *Advances in Neural Information Processing Systems Datasets and Benchmarks Track*.

[126] THUDM. 2026. slime: An LLM Post-Training Framework for RL Scaling. GitHub repository. https://github.com/THUDM/slime

[127] Guiyao Tie, Zeli Zhao, Dingjie Song, Fuyang Wei, Rong Zhou, Yurou Dai, Wen Yin, Zhejian Yang, Jiangyue Yan, Yao Su, et al. 2025. A Survey on Post-training of Large Language Models. *arXiv preprint arXiv:2503.06072* (2025).

[128] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothee Lacroix, Baptiste Roziere, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. LLaMA: Open and Efficient Foundation Language Models. *arXiv preprint arXiv:2302.13971* (2023).

[129] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. *arXiv preprint arXiv:2307.09288* (2023).

[130] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. In *Advances in Neural Information Processing Systems*.

[131] vLLM Team. 2026. vLLM Documentation. Software documentation. https://docs.vllm.ai/

[132] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. 2019. GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding. In *International Conference on Learning Representations*.

[133] Hongyu Wang, Shuming Ma, Li Dong, Shaohan Huang, Dongdong Zhang, Furu Wang, Xing Xie, and Furu Wei. 2024. DeepNet: Scaling Transformers to 1,000 Layers. *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2024).

[134] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *International Conference on Learning Representations*.

[135] Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. 2023. Self-Instruct: Aligning Language Models with Self-Generated Instructions. *Proceedings of ACL* (2023).

[136] Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyan Jiang, et al. 2024. MMLU-Pro: A More Robust and Challenging Multi-Task Language Understanding Benchmark. *arXiv preprint arXiv:2406.01574* (2024).

[137] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems*.

[138] Colin White, Samuel Dooley, Manley Roberts, Arka Pal, Benjamin Feuer, Siddhartha Jain, Ravid Shwartz-Ziv, Neel Jain, Khalid Saifullah, Sreemanti Dey, et al. 2025. LiveBench: A Challenging, Contamination-Free LLM Benchmark. In *International Conference on Learning Representations*.

[139] Thomas Wolf, Lysandre Debut, Victor Sanh, et al. 2020. Transformers: State-of-the-Art Natural Language Processing. In *Proceedings of EMNLP: System Demonstrations*.

[140] Likang Wu, Zhi Zheng, Zhaopeng Qiu, Hao Wang, Hongchao Gu, Tingjia Shen, Chuan Qin, Chen Zhu, Hengshu Zhu, Qi Liu, Hui Xiong, and Enhong Chen. 2024. A Survey on Large Language Models for Recommendation. *World Wide Web* 27, 5 (2024), 60. doi:10.1007/s11280-024-01291-2

[141] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388* (2025).

[142] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. In *Advances in Neural Information Processing Systems*.

[143] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*.

[144] Shukang Yin, Chaoyou Fu, Sirui Zhao, Ke Li, Xing Sun, Tong Xu, and Enhong Chen. 2024. A Survey on Multimodal Large Language Models. *National Science Review* 11, 12 (2024), nwae403.

[145] Xiang Yue et al. 2023. MMMU: A Massive Multi-discipline Multimodal Understanding and Reasoning Benchmark for Expert AGI. *arXiv preprint arXiv:2311.16502* (2023).

[146] Aohan Zeng, Xiao Liu, Zhengxiao Du, Zihan Wang, Hanyu Lai, Ming Ding, Zhuoyi Yang, Yifan Xu, Wendi Zheng, Xiao Xia, et al. 2023. GLM-130B: An Open Bilingual Pre-trained Model. In *International Conference on Learning Representations*.

[147] Biao Zhang and Rico Sennrich. 2019. Root Mean Square Layer Normalization. *Advances in Neural Information Processing Systems* (2019).

[148] Susan Zhang, Stephen Roller, Naman Goyal, et al. 2022. OPT: Open Pre-trained Transformer Language Models. *arXiv preprint arXiv:2205.01068* (2022).

[149] Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q. Weinberger, and Yoav Artzi. 2020. BERTScore: Evaluating Text Generation with BERT. In *International Conference on Learning Representations*.

[150] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. 2023. A Survey of Large Language Models. *arXiv preprint arXiv:2303.18223* (2023).

[151] Lianmin Zheng et al. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *arXiv preprint arXiv:2306.05685* (2023).

[152] Lianmin Zheng et al. 2024. SGLang: Efficient Execution of Structured Language Model Programs. In *Advances in Neural Information Processing Systems*.

[153] Chunting Zhou et al. 2023. LIMA: Less Is More for Alignment. In *Advances in Neural Information Processing Systems*.

[154] Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. 2023. Instruction-Following Evaluation for Large Language Models. *arXiv preprint arXiv:2311.07911* (2023).

[155] Yutao Zhu et al. 2025. Hyper-Connections. *International Conference on Learning Representations* (2025).

[156] Yutao Zhu, Huaying Yuan, Shuting Wang, Jiongnan Liu, Wenhan Liu, Chenlong Deng, Haonan Chen, Zheng Liu, Zhicheng Dou, and Ji-Rong Wen. 2026. Large Language Models for Information Retrieval: A Survey. *ACM Transactions on Information Systems* 44, 1, Article 12 (2026), 54 pages. doi:10.1145/3748304